

OpenBench —用户指南

Open Source Land Surface Model Benchmarking System User's Guide

版本 3.0.0b1 (Beta)

CoLM 陆面模式研发团队

中山大学
大气科学学院

2026 年 6 月

© 2003–2024 戴永久陆面模式研发团队

序

OpenBench (Open Source Land Surface Model Benchmarking System, 开源陆面模式基准评测系统) 是面向陆面模式 (Land Surface Model, LSM) 评估的自动化、跨平台框架。它把”重网格、时间对齐、掩码、指标计算、绘图、出报告”这一整套通常需要为每一对 (模型, 参考数据) 手工拼接的流程, 收敛成一份声明式的 `openbench.yaml` 配置: 一次配置即可对任意数量的变量、模型与参考数据集完成评估、对比与成图。

本指南配套 OpenBench 3.0.0b1 (Beta 版本)。Beta 阶段功能已基本完整, 但部分接口与配置字段在正式版 (3.0.0) 发布前仍可能调整; 涉及变动处本指南会随版本更新。

2026 年 6 月

目录

第一部分 引言	1
1 背景、设计理念与生态	1
1.1 设计理念与适用场景	1
1.1.1 一份配置，多重对比	1
1.1.2 科学一致性与公平比较	1
1.1.3 格点与站点并重	2
1.1.4 增量、可恢复、可复现	2
1.1.5 典型适用场景	2
1.2 与 CoLM 生态及其他模式的关系	2
2 整体架构与运行机制	3
2.1 阶段一：预处理（Preprocess）	3
2.2 阶段二：评估（Evaluate）	3
2.3 阶段三：对比（Compare）	3
2.4 阶段四：报告（Report）	4
2.5 增量缓存与可复现机制	4
3 核心概念、术语与能力	4
3.1 能力总览	5
4 版本、兼容性与阅读指南	5
4.1 本指南的结构与读者对象	6
第二部分 快速入门	7
5 软件环境与安装	7
5.1 安装	7
5.1.1 创建虚拟环境	8
5.1.2 从 PyPI 安装（pip）	8
5.1.3 用 conda / conda-forge 安装	8

5.1.4	从源码安装（开发/本地测试）	9
5.2	周边软件与依赖排错	9
5.2.1	Cartopy 系统库（GEOS / PROJ）编译失败	10
5.2.2	Cartopy / Natural Earth 地图数据下载报错与手动下载	10
5.2.3	netCDF4 / HDF5 相关问题	11
5.2.4	其它可选依赖（PySide6 / f90nml / xhtml2pdf）	12
6	验证安装与首次运行	12
6.1	五分钟跑通第一个评估	13
6.1.1	开跑前：先备好 ref 和 sim 数据	13
6.1.2	init / check / run 常用选项	14
6.2	输出产物速览	15
7	常见问题与运行失败排查	16
7.1	运行失败排查	16
	 第三部分 模式配置	 18
8	配置总体结构与完整示例	18
8.1	配置格式：YAML（取代旧版 namelist）	18
8.2	八个顶层块	18
8.3	_defaults：合并共享设置	19
8.4	!include：跨文件拆分	19
8.5	完整主配置示例	19
9	project 块：全局设置	21
9.1	标识与输出	21
9.1.1	name	21
9.1.2	output_dir	22
9.1.3	years	22
9.2	时空范围	22
9.2.1	lat_range / lon_range	22
9.2.2	min_year_threshold	22

9.3	目标分辨率与对齐	22
9.3.1	tim_res	23
9.3.2	grid_res	24
9.3.3	timezone	24
9.3.4	时间对齐 (time_alignment)	24
9.3.5	重网格后端 (regrid_backend)	24
9.4	运行时与公平性	25
9.4.1	num_cores	25
9.4.2	weight (聚合权重, 三取值)	25
9.4.3	unified_mask	26
9.4.4	generate_report	26
9.4.5	高级开关	26
9.5	分类分组: IGBP / PFT / Köppen	26
10	evaluation 与 reference: 变量与参考数据	27
10.1	evaluation.variables	27
10.2	可用的标准变量名	27
10.3	reference: data_root + 变量绑定	28
10.4	数据集 registry 与命令	28
10.5	grid 与 station 两类参考	28
10.5.1	站点列表 CSV 格式	28
11	simulation 块与 model profile	29
11.1	条目结构	29
11.2	字段取值详解	29
11.2.1	model	29
11.2.2	root_dir	29
11.2.3	data_type	30
11.2.4	data_groupby	30
11.2.5	grid_res / tim_res	30
11.2.6	prefix / suffix	30
11.2.7	fulllist	30

11.2.8 variables	30
11.3 model profile 匹配机制	30
11.4 文件命名: prefix / suffix / data_groupby	31
11.5 逐变量覆盖与”按变量分文件”	31
11.6 用 sim scan 自动生成 simulation 配置	31
11.7 sim scan 推不出 model profile 怎么办	32
12 指标、评分、对比与统计配置	33
12.1 指标 (metrics) 逐项说明	33
12.2 评分 (scores) 逐项说明	35
12.3 对比与统计配置	35
12.3.1 comparison: 跨模型对比图	35
12.3.2 statistics: 独立统计分析	36
13 性能调优与环境变量	38
13.1 io 子块: 高吞吐 IO (完整字段)	38
13.2 dask 子块: 分布式调度 (完整字段)	38
13.3 配置示例	38
13.4 环境变量参考	39
14 GUI 配置向导 (可选)	39
14.1 界面组成	41
14.2 典型操作	41
14.3 常见问题	41
第四部分 实例操作	42
15 案例一: 随包自带的蒸散发 / 能量通量评估 (Initial_test)	42
15.1 一键复现	42
15.2 案例概述与数据组织	43
15.3 配置文件	44
15.4 校验: openbench check	45
15.5 运行: openbench run	46

15.6 结果解读	46
16 案例二：用自己的数据替换	49
16.1 第 1 步：组织数据目录	49
16.2 第 2 步：注册参考数据集	49
16.3 第 3 步：对接模型	49
16.4 第 4 步：改配置并运行	50
17 案例三：扩展对比图与分类分组	50
17.1 配置改动	50
17.2 Taylor 图	51
17.3 IGBP 分类分组	51
17.4 关于 Portrait 与”带错误完成”	52
18 案例四：从旧配置迁移并运行	52
19 案例五：多个真实陆面模式的横向对比	53
19.1 数据与难点：四个模式的命名各不相同	53
19.2 配置：逐模式对接文件约定	54
19.3 校验与运行	55
19.4 结果：综合分数热图	55
19.5 结果：Taylor 图	55
19.6 读法小结	56
20 结果是否合理：读图指南	57
20.1 快速定位结果	57
20.2 怎么判断好坏	57
20.2.1 评分 (scores)	57
20.2.2 指标 (metrics) 方向	57
20.2.3 Taylor 图	58
20.2.4 HeatMap	58
20.2.5 格点空间图	58
20.2.6 站点散点 / CSV	58

第五部分 研发用户指南	59
21 扩展点总览	59
22 注册参考数据与模式 profile	60
22.1 参考数据集的关键字段	60
22.2 格点参考数据集（条目示例）	60
22.3 站点参考数据集（两种形态）	61
22.4 用 ref scan 批量发现并注册	62
22.4.1 扫描遇到无法识别的数据怎么办	63
22.5 追加、移动或删除参考根目录	64
22.6 端到端接入与验收流程	65
22.7 其余 ref 管理命令	66
22.8 注册与定制 model profile	66
22.8.1 model profile 结构与示例	66
23 添加自定义指标、评分、统计与可视化	67
23.1 新增后的测试与验证	68
23.2 添加自定义 statistics 与可视化	69
23.2.1 统计模块	69
23.2.2 图件	69
23.3 自定义滤镜与数据处理	69
24 远程执行、缓存与可复现	70
24.1 两种 HPC 用法	70
24.2 在集群作业脚本里运行（Slurm 最小流程）	70
24.3 GUI 远程（[remote]）	71
24.4 缓存与可复现机制	71
24.4.1 两层缓存	71
24.4.2 用 CLI 管理缓存	72
25 二次分析：用 Python 直接读结果	72
26 参与开发	73

第一部分 引言

1 背景、设计理念与生态

陆面模式（Land Surface Model, LSM）是地球系统模式的核心组件之一，刻画陆地表层的水、能量、碳与人类活动过程。随着模式版本、参数方案、驱动数据与分辨率不断增多，一个绕不开的问题是：**如何客观、可复现地评估一个模式相对观测/参考数据的表现，并在多个模式之间做公平比较？**

传统做法是为每一对（模型, 参考数据）手工编写脚本：读数据、统一网格、对齐时间、做掩码、算指标、画图、汇总。这种做法有三个长期痛点：

- **重复劳动**：每换一个变量、模式或参考，几乎都要重写一遍流水线。
- **口径不一致**：不同脚本对重网格、掩码、加权、缺测处理的处理细节不同，跨模式比较因此不公平。
- **难以复现**：临时脚本、散落的中间文件，半年后自己都难以重跑出同样的结果。

OpenBench（Open Source Land Surface Model Benchmarking System，开源陆面模式基准评测系统）正是为解决这些痛点而生：把整套评估流水线收敛成一份声明式的 `openbench.yaml` 配置，一次配置即可对 **任意数量**的变量、模式与参考数据集完成预处理、评估、对比与成图，并保证口径统一、结果可复现。

1.1 设计理念与适用场景

1.1.1 一份配置，多重对比

OpenBench 把“变量 × 模式 × 参考”视为一个笛卡尔积：在 `openbench.yaml` 里列出要评估的变量、绑定参考、登记模式，OpenBench 会自动展开所有组合并逐对评估（图 1）。每一对的结果独立存放、互不覆盖，因此一份配置就能产出一整套可交叉查阅的评估矩阵。

1.1.2 科学一致性与公平比较

同一套重网格、掩码、加权与指标定义被施加到每一对评估上。尤其是**统一掩码**（unified mask）：OpenBench 把参与对比的各模式的缺测累积成一个共同掩码，使所有模



图 1: 变量 × 模拟 × 参考的笛卡尔积展开为评估任务。

式在完全相同的空间覆盖上被评估——这是跨模式比较公平性的关键，避免”某模式因覆盖范围不同而占便宜”。

1.1.3 格点与站点并重

OpenBench 同等支持**格点**（grid，规则经纬网格 NetCDF）与**站点**（station，逐站点观测）两类参考与模拟。站点评估自动按站点列表逐站抽样、配对、聚合，产出一行一站的指标表。

1.1.4 增量、可恢复、可复现

结果按”配置 + 算法版本 + 输入文件指纹”缓存：命中即跳过，改了什么只重算受影响的部分；中断后可安全续算，不重做已完成的工作，也不产出残缺的汇总。配合落盘的”已解析完整配置”，任何一次运行都可审计、可复现。

1.1.5 典型适用场景

- 单模式离线评估：新版本/新参数方案对观测的整体表现。
- 多模式横向对比：在统一口径下回答”哪个模式在哪个变量上更好”（见第四部分案例五）。
- 版本/敏感性对比：同一模式不同配置之间的差异。
- 面向论文/报告的成图：Taylor、portrait、heat-map 等出版级图件一键生成。

1.2 与 CoLM 生态及其他模式的关系

OpenBench 对应 CoLM 丛书”卷五·评估系统”的定位，但它**独立于具体被评估模式**：内置 22 个模型 profile（CoLM2024、CLM5、NoahMP5、JULES7、VIC5、ERA5-Land、GLDAS2、MATSIRO、CaMa-Flood、WRF 等），可评测任意符合约定的 LSM 输出。换言之

之，无论用的是 CoLM、CLM、Noah-MP 还是再分析产品，都能用同一套 OpenBench 流程进行评估与互比。

2 整体架构与运行机制

OpenBench 的一次评估按四个阶段顺序推进，每个阶段都**增量缓存、可续算**（图 2）。



图 2: OpenBench 四阶段工作流。

2.1 阶段一：预处理（Preprocess）

把每个模式输出与参考数据重网格到统一的目标分辨率（`project.grid_res`）、裁剪到共同时间窗（`project.years` 与时间对齐策略），并按需施加统一掩码。重网格后端可选（`regrid_backend`: 守恒重映射 / CDO / xESMF / 简单插值）。此阶段把”长得各不相同”的输入规整成可逐格/逐站比较的统一形态。

2.2 阶段二：评估（Evaluate）

对每个 (变量, 模拟, 参考) 三元组计算所配置的指标与评分：格点评估产出逐格 NetCDF（每 metric 一个），站点评估产出一行一站的 CSV。指标默认 `bias` / `RMSE` / `correlation`，评分默认 `Overall_Score`，均可在配置中扩展为子集或更多。

2.3 阶段三：对比（Compare）

当 `comparison.enabled` 为真时，跨所有 (模拟, 参考) 对生成共享图与排名：Taylor、portrait、heat-map、SMPI 等（默认仅 Taylor + HeatMap，需显式列出更多）。可选地，`statistics` 阶段还能跑独立统计分析（ANOVA、Mann-Kendall 趋势、三角帽等）。

2.4 阶段四：报告（Report）

把全部产物汇总为 HTML 报告（装 [report] extra 时另出 PDF）。可用 `generate_report: false` 关闭，只产数据与图。

2.5 增量缓存与可复现机制

每个评估任务以”配置 + 算法版本 + 输入文件签名”为键缓存（详见第五部分第 24.4 节）：

- **命中即跳过**：源码（指标/评分算法的内容指纹）、配置或输入文件任一变化即自动失效重算。
- **原子写**：所有输出写临时文件再改名，崩溃不会留下”看似有效”的半截文件。
- **失败不污染**：后处理阶段设有闸门，任一任务失败则不产出可能误导的部分汇总。
- **可续算**：中断后重跑，已完成的安全跳过、未完成的重做；`--force` 可绕过缓存全量重算。

3 核心概念、术语与能力

表 1: 贯穿全书的核心术语

术语	含义
变量 variable	被评估的标准物理量（如 Evapotranspiration、Latent_Heat）
参考 reference	评估基准数据集（grid 或 stn），来自内置 registry 或自注册
模拟 simulation	被评估的模型输出条目（label → 路径与变量映射）
模型 profile	某模式的变量名/单位/文件名约定模板，被 simulation 引用
指标 metric	逐格/逐站的误差度量（bias、RMSE、KGE…）
评分 score	归一化到 0-1 的可比分数（越大越好），如 Overall_Score
对比 comparison	跨模型/参考的汇总图与排名（Taylor、portrait、heat-map…）
分组 groupby	按 IGBP/PFT/Köppen 类别再聚合一份结果
统一掩码 unified mask	跨模型累积的 NaN 掩码，保证空间覆盖一致
时间对齐 time_alignment	模型与参考时间窗的匹配策略（intersection/per_pair/strict）
重网格 regrid	把不同分辨率数据插值到统一目标网格

3.1 能力总览

表 2: OpenBench 能力概览

维度	内容
参考数据集	101 个内置（格点 + 站点），可 list / show / scan / register
模型 profile	22 个内置 (CoLM2024、CLM5、NoahMP5、JULES7、VIC5、ERA5-Land、GLDAS2、MATSIRO、CaMa-Flood、WRF…)
指标 metrics	25+ : bias、percent_bias、RMSE、ubRMSE、CRMSD、MAE、correlation、 R^2 、NSE、KGE、KGESS、index_agreement、rv…
评分 scores	8 个归一化评分: nBiasScore、nRMSEScore、nPhaseScore、nIavScore、nSpatialScore、nSeasonalityScore、index_agreement、Overall_Score
统计模块	17 : ANOVA、Correlation、Mann-Kendall、Three-Cornered-Hat、Hellinger、PLSR、FDR…
对比可视化	Taylor、Target、Portrait、HeatMap、Parallel Coordinates、KDE、Ridgeline、Whisker、Diff、SMPI、Relative Score、Mann-Kendall …
分类分组	IGBP (17 类)、PFT (16 类)、Köppen (30 类)，掩码随包自带
接口	CLI (run/check/init/ref/sim/model/cache/migrate/smoke-test/gui) + 可选 GUI 向导

4 版本、兼容性与阅读指南

说明：本手册适用版本：**OpenBench 3.0.0b1 (Beta)**

与 v2.x 主要差异：单一 openbench.yaml 取代旧 Fortran namelist 多文件；命令行统一为 openbench 子命令 (run/check/init/ref/sim/model…)；评估结果与 v2.x **数值一致**。

旧配置怎么办：用 openbench migrate old.nml -o openbench.yaml (或 .json) 一键转换，见第四部分案例四。

本指南配套 OpenBench 3.0.0b1 (**Beta**)。Beta 阶段功能已基本完整，但部分接口与配置字段在正式版 3.0.0 发布前仍可能调整。版本号遵循 PEP 440：3.0.0b1 即”3.0.0 的第 1 个 Beta”，定稿即去后缀为 3.0.0。OpenBench 3.0 的评估结果与 2.x 数值一致。

说明：Beta 预发布版的安装：当 PyPI 上仅有 Beta 时 pip install colm-openbench 会直接装上；一旦发布正式版，安装 Beta 需 pip install --pre colm-openbench 或显式指定版本号。

4.1 本指南的结构与读者对象

本指南分五部分，按读者层次组织：

- **第一部分·引言**（本部分）：概念、架构与术语，建立整体认识。
- **第二部分·快速入门**：面向初次使用者——安装、周边软件排错、自检、五分钟跑通第一个评估。
- **第三部分·模式配置**：作为配置参考手册，逐块讲清 `openbench.yaml` 的所有字段与取值。
- **第四部分·实例操作**：可复现的完整案例（含真实多模式对比），照搬后按需替换数据与模型。
- **第五部分·研发用户指南**：面向需要扩展功能、接入自有数据/模型、做二次分析或参与开发的用户。

初级用户从第二部分开始；长期使用者把第三部分当手册查阅；研发用户重点读第五部分。各章之间以交叉引用（如“见第 9.5 节”）相互衔接，便于跳读。

第二部分 快速入门

本部分面向初次使用者：先备好软件环境并安装，处理好 Cartopy 等”重”依赖的常见坑，用内置样例自检，再用五分钟跑通第一个评估，最后认识输出产物。

5 软件环境与安装

OpenBench 是纯 Python 包，跨 Linux / macOS / Windows 运行，要求 **Python 3.10 或更高**。核心科学栈较”重”，建议在干净虚拟环境（venv / conda）中安装，避免与系统 Python 冲突。

表 3: 依赖一览

类别	内容
核心（自动装）	xarray、numpy、scipy、pandas、netCDF4、matplotlib、cartopy、dask、joblib、flox、PyYAML、Jinja2、click、tqdm、packaging
[gui]	PySide6（图形配置向导）
[remote]	paramiko、cryptography（SSH 远程执行）
[report]	xhtml2pdf（PDF 报告；不装则仅出 HTML）
[migration]	f90nml（迁移旧 Fortran NML 配置）
[all]	以上可选项全部

说明：对 HPC/服务器，核心安装（仅 CLI）即可，无需 [gui]。可选项缺失时 OpenBench 会给出友好提示而非崩溃（如未装 [report] 时自动跳过 PDF、只出 HTML）。

5.1 安装

OpenBench 以 Python 包 colm-openbench 发布，可用 **pip** (§5.1.2) 或 **conda / conda-forge** (§5.1.3) 安装。核心科学栈较”重”，建议装在干净的虚拟环境（venv 或 conda）里，避免污染系统 Python。若平台缺 **Cartopy** 等地理空间库的预编译包，建议直接用 **conda**（见 §5.1.3），可省去本地编译。

说明：命令书写约定（贯穿全书）：代码块里 # 之后是注释，说明用、**不要照打**；<...>（尖括号）是占位符，需替换成你自己的值（如把 <work_dir> 换成真实路径）；而 [...]（方括号）是命令的一部分、**要原样输入**——它是 pip 的”可选项（extras）”

语法: `pip install "colm-openbench[gui]"` 表示装核心包的同时再额外装上名为 `gui` 的那组可选依赖。含方括号时整条用引号 `"..."` 括起, 以免 `zsh` 等 shell 把方括号当通配符报错。

5.1.1 创建虚拟环境

```
python -m venv .venv          # 在 .venv/ 下新建一个隔离的虚拟环境
source .venv/bin/activate      # 激活它 (Windows: .venv\Scripts\activate)
python -m pip install --upgrade pip  # 顺手升级 pip 本身 (推荐)
```

5.1.2 从 PyPI 安装 (pip)

最简——只装核心 (命令行评估, 最适合 HPC / 无图形界面的服务器):

```
pip install colm-openbench      # 只装核心 CLI, 不含任何可选项
```

需要额外功能时, 在包名后加方括号”可选项” (可逗号组合, 如 `[gui,report]`):

```
pip install "colm-openbench[gui]"      # +图形配置向导 (需 PySide6/Qt)
pip install "colm-openbench[remote]"    # +远程 SSH 执行 (需 paramiko)
pip install "colm-openbench[report]"    # +PDF/HTML 报告 (需 xhtml2pdf)
pip install "colm-openbench[migration]"  # +旧版 Fortran namelist 迁移 (需 f90nm1)
pip install "colm-openbench[all]"        # +以上全部
pip install "colm-openbench[gui,report]" # 也可只挑几个, 逗号分隔、无空格
```

各”可选项”引入的具体依赖见前面的依赖一览表 (表 3, §5)。按需选择: HPC/纯命令行用核心即可, 要图形界面加 `[gui]`, 要成品报告加 `[report]`, 全都要用 `[all]`。

说明: 当前为 Beta 预发布版本 3.0.0b1。PyPI 上若同时有正式版与 Beta, `pip` 默认装正式版; 显式装 Beta 用 `pip install --pre colm-openbench` 或 `pip install colm-openbench==3.0.0b1`。

5.1.3 用 conda / conda-forge 安装

OpenBench 自带一份 conda-forge 配方 (仓库 `conda/meta.yaml`, `noarch`: 从 PyPI 取源, 由 conda 解析 `Cartopy/NetCDF/PROJ` 等地理空间依赖)。conda 用户有三条路径:

(a) 从 conda-forge 直接安装 (发布后)。正式上架 conda-forge 后, 一条命令即可——conda 会一并装好 `Cartopy/NetCDF/PROJ` 等”重”依赖, 免去本地编译:

```
conda install -c conda-forge colm-openbench # 连地理空间重依赖一并装好
mamba install -c conda-forge colm-openbench # 或用更快的 mamba (等价)
```

注意：当前为 Beta 预发布版本 3.0.0b1，尚未上架 **conda-forge**。上架前请用下面 (b) 的“conda 装依赖 + pip 装本体”，或 (c) 从随包配方本地构建。

(b) conda 解依赖 + pip 装本体 (Beta 推荐)。若平台没有 Cartopy 等的预编译 wheel，先用 conda-forge 装地理空间科学栈，再用 pip 装 OpenBench（可带 [gui] 等可选项）：

```
mamba create -n openbench -c conda-forge python=3.12 cartopy netcdf4 #
    建环境+装地理空间依赖
mamba activate openbench # 激活该环境
pip install colm-openbench # 再用 pip 装本体 (或 "
    colm-openbench[all]")
```

(c) 从随包配方本地构建 conda 包 (可选)。想现在就得到一个本地 conda 包，可用仓库里的配方构建：

```
conda install -n base conda-build # 装 conda-build 工具
conda build conda/ # 用 conda/meta.yaml 本地
    构建
conda install -c local colm-openbench # 从本地通道安装刚构建的包
```

5.1.4 从源码安装 (开发/本地测试)

```
git clone https://github.com/CoLM-SYSU/OpenBench.git # 克隆源码仓库
cd OpenBench # 进入仓库目录
python -m pip install -e ".[dev]" # 可编辑安装 + 开发
    依赖 (pytest/ruff)
```

说明：分类分组 (IGBP/PFT/Köppen) 所需的分类掩码已随包发行 (压缩后约 1.1 MB)，无需额外下载。

5.2 周边软件与依赖排错

陆面评测涉及若干“重”科学库，最常见的安装/运行问题集中在 Cartopy 与 NetCDF。本节给出可直接照做的解决方案。

5.2.1 Cartopy 系统库 (GEOS / PROJ) 编译失败

Cartopy 依赖系统级的 GEOS、PROJ 库。若 `pip install` 阶段报编译错误，先装系统依赖再重装：

```
# Debian/Ubuntu
sudo apt-get install -y libgeos-dev libproj-dev proj-bin

# macOS (Homebrew)
brew install geos proj
```

或直接用 conda 安装 Cartopy (见上一节)，跳过本地编译。

5.2.2 Cartopy / Natural Earth 地图数据下载报错与手动下载

现象。安装本身成功，但在绘图阶段 (`comparison / figures`) 首次需要海岸线、陆地、国界等底图时，Cartopy 会按需联网下载 Natural Earth 矢量数据。在离线、内网或被防火墙限制的机器 (HPC 计算节点尤甚) 上，会抛出类似下列报错而中断绘图：

在可联网的机器上，表现为下载提示 (下载成功，绘图继续)：

```
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturlaearth.s3.amazonaws.com/110m_physical/ne_110m_land.zip
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturlaearth.s3.amazonaws.com/110m_physical/ne_110m_coastline.
  zip
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturlaearth.s3.amazonaws.com/110m_physical/ne_110m_lakes.zip
.../cartopy/io/__init__.py:241: DownloadWarning: Downloading:
  https://naturlaearth.s3.amazonaws.com/110m_physical/
  ne_110m_rivers_lake_centerlines.zip
```

在离线 / 内网 / 防火墙机器上，同样的下载会失败并中断绘图，报错形如：

```
urllib.error.URLError: <urlopen error [Errno 110] Connection timed out>
# 或
HTTPError: HTTP Error 404: Not Found
# 或
ssl.SSLCertVerificationError: certificate verify failed
```

原因。这不是 OpenBench 的 bug，而是 Cartopy 在运行时按需获取底图资源的机制在无外网环境下失败。OpenBench 默认底图请求的就是上面 4 个 110m_physical 数据集 (land、coastline、lakes、rivers_lake_centerlines)。

解决：手动下载并放到 Cartopy 数据目录。

第 1 步，确认 Cartopy 期望的数据目录：

```
python -c "import cartopy; print(cartopy.config['data_dir'])"
# 常见路径：
#   Linux/macOS: ~/.local/share/cartopy
#   也可由环境变量 CARTOPY_DATA_DIR 指定
```

第 2 步，在一台可联网的机器上下载 OpenBench 默认底图所需的 4 个 110m_physical 数据集（按需评估更精细底图时，可同样补 50m/10m 版本）。Cartopy 默认源：

```
https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_land.zip
https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_coastline.zip
https://naturalearth.s3.amazonaws.com/110m_physical/ne_110m_lakes.zip
https://naturalearth.s3.amazonaws.com/110m_physical/
    ne_110m_rivers_lake_centerlines.zip
官方页面：https://www.naturalearthdata.com/downloads/
```

第 3 步，解压并放到 physical 子目录（目录结构必须与分辨率/类别一致）：

```
DATA_DIR=$(python -c "import cartopy; print(cartopy.config['data_dir'])")
DEST="$DATA_DIR/shapefiles/natural_earth/physical"
mkdir -p "$DEST"
for n in ne_110m_land ne_110m_coastline ne_110m_lakes
    ne_110m_rivers_lake_centerlines; do
    unzip -o "$n.zip" -d "$DEST"
done
```

第 4 步（可选），也可用 Cartopy 自带的下载脚本一次性拉取（在可联网机器上）：

```
python -m cartopy.feature.download physical cultural --output "$DATA_DIR"
```

说明：放好后再次运行 OpenBench，绘图阶段不再联网即可成图。若用共享 CARTOPY_DATA_DIR，可让全集群复用同一份底图。

5.2.3 netCDF4 / HDF5 相关问题

两类常见问题：

(1) HDF5 文件锁（NFS / 共享盘）。在某些 NFS 或集群共享文件系统上，HDF5 的文件锁会导致读 NetCDF 时报错或卡住（形如 HDF5 error: unable to lock file）。临时关闭文件锁：

```
export HDF5_USE_FILE_LOCKING=FALSE
openbench run openbench.yaml
```

(2) numpy / netCDF4 ABI 不匹配。若导入时报 `numpy.ndarray size changed` 之类的二进制兼容警告/错误，多为 `numpy` 与 `netCDF4`/编译扩展版本不配。重装对齐即可：

```
pip install --force-reinstall --no-cache-dir numpy netCDF4
```

5.2.4 其它可选依赖（PySide6 / f90nml / xhtml2pdf）

这些依赖仅在用到对应功能时才需要，缺失时 `OpenBench` 会给出明确提示：

- **PySide6** ([gui]) ：运行 `openbench gui` 需要。缺失会提示 `pip install "colm-openbench[gui]"`。
- **f90nml** ([migration])：迁移 `.nml` 旧配置 (`openbench migrate xxx.nml`) 需要；迁移 JSON 不需要。
- **xhtml2pdf** ([report])：生成 PDF 报告需要。不装则评估照常完成、仅出 HTML 报告，不报错。

6 验证安装与首次运行

安装后、准备自己的数据前，强烈建议先用**内置样例**自检——它不依赖你的任何数据，能一步确认”装好了、能跑通”：

```
openbench smoke-test          # 解包内置 Initial_test 样例并跑 openbench
    check (快)
openbench smoke-test --run      # 进一步跑一次完整评估 (含成图)
openbench smoke-test --run --keep # 跑完保留工作目录，便于查看产物
```

命令会打印工作目录与配置路径；`--run` 结束时还会列出生成的关键结果产物（并校验其确实生成）。跑完后在工作目录的 `output/openbench_initial_test_smoke/` 下，重点看这几处：

- **综合分数热图**：`comparisons/HeatMap/scenarios_Overall_Score_comparison_heatmap.jpg`（配套数据 `..._comparison.csv`）。
- **空间分布图**：`scores/< 变量 >_..._Overall_Score.jpg`（格点）。
- **站点结果**：`metrics/、scores/` 下的 `< 变量 >_stn_..._evaluations.csv`（一行一站）。

若 `smoke-test` 通过，说明环境就绪。这个样例的完整拆解见第四部分案例一。

说明：`--run` 的完整评估在干净机器上首次绘图可能触发 Cartopy 联网下载底图（见第 5.2.2 节）。

6.1 五分钟跑通第一个评估

OpenBench 的全部命令一览（`openbench --help`）：

```
Commands:
  run          Run evaluation from a config file.
  check        Validate config file and check data availability.
  smoke-test   Run the bundled Initial_test smoke fixture.
  init         Interactively generate an openbench.yaml config file.
  ref          Manage reference datasets.
  sim          Manage simulation outputs.
  model        Manage model profiles.
  migrate      Convert old JSON/NML config to unified YAML.
  cache        Manage OpenBench runtime caches.
  gui          Launch the OpenBench graphical interface.
  version      Show version information.
```

6.1.1 开跑前：先备好 ref 和 sim 数据

§6 的 `smoke-test` 已用内置样例验证了环境；要评估你自己的模式，则必须先在本机备好两类数据：

- **参考数据（reference）**：观测/基准，格点 NetCDF 或站点数据。
- **模拟数据（simulation）**：你要被评估的模式输出。

OpenBench 本身不含这些数据，只带一份描述“如何读取已知数据集”的 registry——所以数据要你自己提供，下面的 `init` 只负责发现、登记、生成配置（不复制数据）。

注意：要先单独 `scan` 吗？不用。`openbench init` 会当场问你“参考数据根”和“模拟数据根”，然后内部自动扫描并注册（相当于替你跑了 `ref scan / sim scan`）。独立的 `openbench ref scan < 目录 > / openbench sim scan < 目录 >` 是给手写配置、批量/CI 预注册、或日后新增数据补扫用的——首次走 `init` 不必单独 `scan`。扫描时若遇到无法识别的数据（变量歧义、布局非标准等），处理办法见第五部分第 22.4.1 节。

入门只需 `init` → `check` → `run` 三步：生成配置 → 校验 → 运行。

```

openbench init          # 1) 交互式生成配置：会问你 ref 根 + sim
    根
openbench check openbench.yaml # 2) 校验配置 + 数据可用性
openbench run   openbench.yaml # 3) 评估 + 成图 + 出报告

```

各步做什么：

- **init**：交互式向导——先问你”参考数据根”与”模拟数据根”，自动扫描并注册其中的数据集，再依次选择变量、为每个变量挑参考、确认模式 profile，生成一份可直接运行的 openbench.yaml（不想交互可手写或用 ref scan/sim scan 拼，见第三、五部分）。
- **check**：**运行前必做**。逐条报告每个 (变量, 参考) 是否定位成功、共展开多少评估任务、各项选项取值；能在真正开跑前发现路径/变量名/单位的问题。
- **run**：依次完成预处理、评估、对比、报告。改了配置再跑只重算受影响部分；--force 全量重算。

结果落在 output/<project_name>/；首先打开 reports/report.html。

6.1.2 init / check / run 常用选项

三个核心命令的常用选项：

openbench init（交互生成配置，可用选项预设扫描行为）：

选项	含义
--ref-root PATH	参考数据根（init 扫描/更新预检用）
--refresh-ref	init 时直接刷新参考 catalog，不再询问
--no-ref-check	跳过参考 catalog 扫描/更新预检
--sim-root PATH	要扫描的模拟数据根（可重复）
--sim-model NAME	模拟的模型 profile 名，或 auto（默认 auto）
--sim-case-depth N	扫描模拟 case 的目录深度（0-10，默认 5）
--sim-case-pattern GLOB	只纳入匹配该 glob 的 case 标签
--sim-exclude GLOB	排除的模拟目录名/glob（可重复）
--sim-climatology auto off year month	检测或强制模拟的气候态 tim_res 处理

说明：init 会每次都重新扫描参考吗？不会。是否扫描取决于参考 catalog 的状态：

- **首次** (catalog 缺失或为空): **必须扫描一次**, 这是 init 继续的前提。
- **已注册过** (catalog 非空): init **不会自动重扫**, 只问一句 “Reference catalog exists. Refresh it from the data root now?”, **默认为「否」**——直接回车即跳过扫描、复用已注册的参考。

注册结果持久化在用户目录 `~/.openbench/references/reference_catalog.yaml`, 参考根目录也会被记住, 因此之后每次 init 都能复用、无需重扫。要控制这一行为: `--refresh-ref` 强制重扫刷新 (数据有更新时用)、`--no-ref-check` 完全跳过参考预检 (已注册、想最快进入配置生成时用)、`--ref-root PATH` 直接指定参考根。

openbench check:

- `--comparison-only`: 按”仅对比”模式校验, 跳过本地模拟根检查。
- `--strict-reference` (`--strict`): 把低可信参考元数据视为错误 (不改 YAML)。
- `--variable VAR` (可重复): 只校验指定变量。

openbench run: `--force` (忽略缓存全量重算)、`--comparison-only` (仅对已有结果重跑对比)、`--dry-run` (只检查不执行)、`--cores N`、`--variable VAR` (可重复)、`--output-dir DIR`、`--dump-config` (导出中间/调试配置)。

6.2 输出产物速览

一次成功的 run 产出大致如下 (详见第四部分案例与第七节结果解读):

```
output/<project_name>/
+-- config.yaml      # 本次实际使用的完整配置 (含解析后的默认值)
+-- run.log          # 完整运行日志
+-- data/            # 预处理后的 sim/ref NetCDF (重网格、裁剪后)
+-- metrics/         # 每 (变量, 参考, 模拟, metric) 一个 NC (格点) / CSV (
    站点) + 图
+-- scores/          # 归一化评分 (同 metrics 命名) + 图
+-- comparisons/     # 跨模型对比图与数据 (comparison.enabled=true
    时)
+-- statistics/      # 统计分析 (statistics.enabled 且有 items 时)
+-- reports/         # report.html (装 [report] 时另有 report.pdf)
`-- .openbench_cache.json # 增量缓存索引
```

首次查看顺序: `reports/report.html` → `comparisons/` (跨模型一览) → `metrics/`、`scores/` (逐对细看)。

7 常见问题与运行失败排查

- `pip install colm-openbench` 找不到版本？当前是 Beta 预发布，用 `pip install --pre colm-openbench` 或显式 `==3.0.0b1`。
- **Cartopy 装不上 / 绘图报 Natural Earth 下载错误？** 见第 5.2.1、5.2.2 节。
- **读 NetCDF 报 HDF5 文件锁错误？** `export HDF5_USE_FILE_LOCKING=FALSE`（见第 5.2.3 节）。
- **check 报某变量没有参考？** 检查 reference 里变量名大小写、数据集名是否存在（`openbench ref show < 名 >`）、`data_root/`路径是否正确。
- **没有自己的数据想先体验？** 直接 `openbench smoke-test --run --keep`（内置样例）。
- **groupby (IGBP/PFT/气候带) 要额外下载掩码吗？** 不需要，掩码随包自带。

7.1 运行失败排查

`check` 是静态校验（配置/路径/数据可达），通过不代表 `run` 一定成功——运行期还可能因数据内容、绘图、内存等出问题。常见场景与对策：

现象	对策
<code>check</code> 通过但 <code>run</code> 失败	看 <code>run.log</code> 末尾的报错；多为变量名/单位不匹配、文件实际内容缺该变量、内存不足（降 <code>num_cores</code> 或开 <code>dask</code> ）。
对比某子项失败（如 Portrait）	多为数据覆盖不足（如季节缺有限值）；OpenBench 会单项报错、其余照常产出、命令非零退出（见案例三）。补全数据或去掉该图种。
跑完没有图	检查 <code>comparison.enabled: true</code> 且显式列了 <code>items</code> ；逐对图在 <code>metrics/</code> 、 <code>scores/</code> 下。
绘图阶段卡住/报下载错	Cartopy 离线取底图失败——按第 5.2.2 节手动放好 Natural Earth。
站点全被跳过/找不到	站点 CSV 的 <code>DIR</code> 路径错或相对路径； <code>min_year_threshold</code> 过大把短记录站点滤掉了。
改了配置却没重算	改的不是被纳入指纹的输入；用 <code>--force</code> 或删除该 <code>run</code> 的 <code>.openbench_cache.json</code> （见第五部分第 24.4 节）。
区域/站点/全球结果对不上	三者口径不同： <code>lat_range/lon_range</code> 决定区域、站点按点配对、权重（ <code>weight</code> ）影响聚合——确认口径一致再比较。
HDF5 文件锁报错（NFS）	<code>export HDF5_USE_FILE_LOCKING=FALSE</code> 。

说明：排查第一步永远是看输出目录下的 `run.log`——它按阶段记录了每个任务的成功/失败与原因。

第三部分 模式配置

本部分是 OpenBench 的配置参考手册，逐块讲清 `openbench.yaml` 的所有字段、取值与常见坑。书写约定：字段名用 `block.field` 表示；省略某字段时 OpenBench 代入下文标注的默认值；代码块里 `<...>` 为占位符。

8 配置总体结构与完整示例

8.1 配置格式：YAML（取代旧版 `namelist`）

OpenBench 3.x 用单一 **YAML 文件** (`openbench.yaml`) 描述一次评估，取代了 2.x 时代的 Fortran **namelist** (`.nml`) 多文件方案。两者的对应关系：旧 `namelist` 里 `general/ref/sim/metrics` 等分散的字段，现统一收敛到下文的八个 YAML 顶层块。已有 `namelist`（或 JSON）旧配置可用 `openbench migrate` 一键转换为 YAML（见第四部分案例四）。本部分只讲新的 YAML 配置；若仍在维护旧 `namelist`，请先迁移再参照本手册。

8.2 八个顶层块

一份配置的骨架如下，八个顶层块各司其职：

```
project:      # 全局：标识、时空范围、目标分辨率、并行、对齐、掩码、权重、报告
evaluation:   # 评估哪些变量
reference:    # data_root + 每个变量绑定哪个(些)参考数据集
simulation:   # 被评估的模型输出（可多个，label -> 条目）
metrics:     # 计算哪些指标                （省略 -> [bias, RMSE, correlation]）
scores:      # 计算哪些归一化评分          （省略 -> [Overall_Score]）
comparison:  # 跨模型对比图开关与项        （省略 items -> [Taylor_Diagram, HeatMap]）
statistics:  # 统计分析开关与项            （省略 items -> 空集，即便 enabled 也不跑）
```

其中 `project`、`evaluation`、`reference`、`simulation` 为**必填**；其余四块可省略并走默认。

8.3 _defaults: 合并共享设置

在 `simulation` 下写一个 `_defaults` 块，其字段会并入每个模拟条目，省去重复：

```
simulation:
  _defaults:
    data_type: grid
    tim_res: Month
    data_groupby: Month
  run_a: {model: CoLM2024, root_dir: /data/A}
  run_b: {model: CLM5,      root_dir: /data/B}
```

8.4 !include: 跨文件拆分

把一段配置拆到单独文件，用 `!include` 引入，便于复用与版本管理：

```
simulation: !include sims/all_models.yaml
```

说明：已弃用的 `options` 块：旧配置里写在顶层 `options:` 下的字段会被 OpenBench 自动迁移进 `project` 并给出警告。新配置请直接写在 `project` 下，或运行 `openbench migrate`。

8.5 完整主配置示例

下面是一份八块齐全、带注释的主 `openbench.yaml`，可作为模板复制后按需修改；本节后续各章再逐块详解每个字段。该模板已通过 OpenBench 配置加载器校验，附在 `examples/openbench-full.yaml`。

OpenBench 附带 4 份示例配置，用途各异，运行前请看清标注（每份文件开头也有同样标注）：

表 4: 随附示例配置

文件	状态	说明
<code>openbench-smoke.yaml</code>	可运行（经 <code>smoke-test</code> ）	内置样例模板，用 <code>openbench smoke-test --run</code> 跑，勿手动跑带占位符的原文件
<code>openbench-full.yaml</code>	仅模板 / 需用户数据	八块齐全的注释参考模板
<code>openbench-min.yaml</code>	仅模板 / 需用户数据	最小配置骨架
<code>multimodel.yaml</code>	需用户数据 / 路径占位	四模式对比（案例五），路径需替换

注意：除 `openbench-smoke.yaml`（经 `smoke-test` 驱动）外，其余示例的路径均为占位符，直接 `run` 会因找不到数据而失败——请先替换为本机路径。

```
project:
  # --- 标识（必填） ---
  name: my_benchmark                                # 输出落在 output_dir/name/
  output_dir: ./output
  years: [2004, 2005]                                # 评估时间窗，含端点
  # --- 时空范围 ---
  lat_range: [-90, 90]
  lon_range: [-180, 180]
  min_year_threshold: 1                                # 站点：有效年限 < N 的站点剔除
  # --- 目标分辨率与对齐 ---
  tim_res: Month                                    # Month | Day | Hour | Year
  grid_res: 2.0                                    # 重网格到的目标分辨率（度）
  time_alignment: intersection                    # intersection | per_pair | strict
  regrid_backend: openbench_conservative
  # --- 运行时与公平性 ---
  num_cores: 4                                    # null/0 -> 全部 CPU
  weight: area                                    # area | mass | none（省略亦为 area）
  unified_mask: true
  generate_report: true
  # --- 分类分组（默认 false） ---
  IGBP_groupby: false
  PFT_groupby: false
  climate_zone_groupby: false
  # io: {...}    dask: {...}                        # 可选子块，见性能调优一章

evaluation:
  variables: [Evapotranspiration, Latent_Heat]

reference:
  data_root: /data/Reference
  Evapotranspiration: ET_Xu_etal_2025_LowRes        # 单个参考
  Latent_Heat: FLUXCOM                             # 也可写列表做多参考对比

simulation:
  _defaults:                                        # 并入下面每个条目（可省）
    data_type: grid
    tim_res: Month
    data_groupby: Month
  CoLM2024_run:
    model: CoLM2024                                # 匹配内置 profile -> 自动变量映射
    root_dir: /data/Simulation/CoLM2024
    prefix: hist_                                    # 文件 <prefix>YYYY-MM.nc
    variables:                                       # 覆盖 profile 的变量名/单位
      Latent_Heat: {varname: f_lfevpa, varunit: W m-2}
```

```

metrics: [bias, RMSE, correlation, KGE]      # 省略 -> [bias, RMSE,
correlation]
scores:  [nBiasScore, nRMSEScore, Overall_Score]  # 省略 -> [
Overall_Score]

comparison:
  enabled: true
  items: [HeatMap, Taylor_Diagram]    # 省略 items 即这两种

statistics:
  enabled: false                      # 启用需 enabled: true 且显式列出
  items

```

9 project 块：全局设置

project 块汇集一次评估的**全局设置**：标识与输出、时空范围、目标分辨率与对齐、运行时与公平性。下面分四组逐字段说明（必填项见各组标注）。

9.1 标识与输出

表 5: 标识字段（必填）

字段	取值	含义
name	字符串	项目名；输出落在 output_dir/name/
output_dir	路径	输出根目录（相对 cwd 或绝对）
years	[起, 止]	评估时间窗，含端点，恰好两个整数

9.1.1 name

项目名，决定输出子目录 output_dir/name/。须为单一目录名（不含路径分隔符或空格）；同名重跑会复用同一输出目录并触发增量缓存（改了配置只重算受影响部分）。换个 name 即得到一份全新、互不干扰的输出。

9.1.2 output_dir

输出根目录,可相对(相对当前工作目录)或绝对。所有产物(data/metrics/scores/figures/comparisons/reports)都落在其下的 name/ 里。

9.1.3 years

形如 [起, 止] 的两个整数, 含端点。它是评估时间窗的”请求范围”; 实际使用范围还要与各数据集的可用年份、以及 time_alignment 取交集后决定。若请求年份某数据完全不覆盖, 该对会被跳过并在日志/check 中提示。

9.2 时空范围

表 6: 时空范围字段

字段	默认	含义
lat_range	[-90,90]	纬度范围
lon_range	[-180,180]	经度范围
min_year_threshold	3	站点参考: 有效年限 $< N$ 的站点剔除 (仅站点)

9.2.1 lat_range / lon_range

形如 [min, max] 的经纬度范围, 默认全球 ([-90,90] / [-180,180])。OpenBench 据此裁剪所有格点数据与站点 (范围外的站点会被排除)。做区域评估 (如只看东亚、只看某流域) 只需缩小这两个范围, 无需改动数据本身。经度约定取决于数据 ([-180,180] 或 [0,360]); 与数据保持一致即可。

9.2.2 min_year_threshold

默认 3, 仅对站点参考生效。一个站点的有效观测年限若少于该阈值, 则该站点被剔除、不参与评估。用于过滤记录过短、统计不稳健的站点; 调小 (如 1) 可保留更多短记录站点 (内置样例就用 1), 调大则更严格。

9.3 目标分辨率与对齐

这组字段决定所有数据被统一到”公共尺度”, 影响评估、对比、统计所有阶段。

表 7: 分辨率与对齐字段

字段	取值 / 默认	含义
tim_res	Month Day Hour Year	目标时间分辨率
grid_res	浮点（度），如 0.5	目标空间分辨率（重网格到此）
timezone	浮点（小时）	时区偏移（仅小时/日尺度有意义）
time_alignment	intersection(默认)	见下
regrid_backend	openbench_conservative(默认)	见下

9.3.1 tim_res

目标**时间**分辨率，所有数据被聚合/对齐到此尺度。合法值：Year、Month、Day、Hour、3Hour、6Hour、8Day（以及 3month 之类多月写法），外加两个**气候态**取值 climatology-month、climatology-year（见下方“气候态评估模式”）。需与评估意图一致：月尺度评估写 Month。若源数据比目标更细，OpenBench 做时间聚合；更粗则可能无法满足而报错。

气候态评估模式(climatology) 把 tim_res 设为 climatology-year 或 climatology-month，即进入**气候态**评估——先把整段时间按（时长加权）平均成气候态，再与参考比较，而非逐时刻的时间序列评估：

- climatology-year: **年气候态**——整段时间平均成单个时间点（多年平均态）。
- climatology-month: **月气候态**——按月分组平均成 12 个时间点（多年平均季节循环）。

```
project:
  years: [2001, 2010]
  tim_res: climatology-month      # 评估多年平均的季节循环
```

何时用：关注模式对长期平均态或平均季节循环的刻画（而非逐月时间序列的逐点吻合）时。

注意：气候态下部分**依赖时间维**的指标不可用：年气候态只有 1 个时间点，相关系数、趋势、相位等无法计算，OpenBench 会自动跳过这些指标并在日志中提示“Unsupported climatology metric(s)”；月气候态有 12 点，可算季节相关等，但仍不适用年际变率类指标。请据评估目的选指标。

说明：时间分辨率（含气候态）在 v3 统一由 `project.tim_res` 指定，它同时驱动对比阶段的内部 `compare_tim_res`。旧版把 `tim_res/grid_res/weight` 等写在 `comparison` 下的做法已**弃用**——OpenBench 仍会把它们自动迁移进 `project` 并给出警告，建议直接写在 `project` 或运行 `openbench migrate`。

9.3.2 grid_res

目标**空间**分辨率（单位：度，如 0.5、0.25、2.0）。所有格点数据都**重网格**到此分辨率后再比较——这是把不同原生分辨率的模式/参考放到同一格点上的关键。一般取参与对比各方中较粗者，避免把粗数据强行插值到过细网格。站点评估不受此影响（按站点配对）。

9.3.3 timezone

时区偏移（小时，浮点）。仅在**小时/日**尺度评估时有意义，用于把模式与观测的时间对齐到同一时区；月及以上尺度可忽略。

9.3.4 时间对齐 (time_alignment)

三选一，决定模型与参考时间窗如何匹配：

- `intersection`（默认）：取两者时间窗**交集**评估。最稳健，推荐多数场景。
- `per_pair`：每个 (sim, ref) 对**各自**对齐，为每个 sim 复制一份对齐后的 ref。适合不同 sim 时间覆盖差异较大、希望各取所长时。
- `strict`：要求时间窗**严格一致**，否则报错。适合要求数据严格可比的严谨场景。

9.3.5 重网格后端 (regrid_backend)

四选一，决定如何把数据插值到 `grid_res`：

- `openbench_conservative`（默认）：OpenBench 内置的守恒重映射，无需外部依赖，保通量守恒，适合 ET、潜热、降水等**通量类**变量，是推荐默认。
- `cdo_remapcon`：调用系统 CDO 的守恒重映射（需装 CDO）。与既有 CDO workflow 一致时可用。
- `xesmf_conservative`：调用 xESMF 的守恒重映射（需装 xESMF/ESMF）。

- `basic_interpolation`: 简单（双线性等）插值，最快但不守恒；适合状态量或快速试跑，通量类变量慎用。

9.4 运行时与公平性

表 8: 运行时字段

字段	取值 / 默认	含义
<code>num_cores</code>	整数 / <code>None</code>	并行核数 (<code>None</code> / <code>0</code> → 全部 CPU)
<code>weight</code>	<code>area</code> <code>mass</code> <code>none</code>	空间聚合权重, 省略 → area
<code>unified_mask</code>	<code>true</code> (默认)	跨模型累积统一掩码 (公平性)
<code>generate_report</code>	<code>true</code> (默认)	出 HTML/PDF 报告

9.4.1 `num_cores`

并行核数。`None`（省略）或 `0` 表示用全部 CPU；设具体整数可限制占用（HPC 共享节点上建议显式设定）。它控制变量/任务级并行；更细的分布式调度见 `dask`。

9.4.2 `weight`（聚合权重，三取值）

控制把逐格点结果聚合成一个数时如何加权。它不改变 `metrics/`、`scores/` 下的逐格点结果，只影响下游聚合（如分类表的“Overall”列）：

- **area（省略时的实际默认）**：余弦纬度面积权 $\cos \phi$ ，对应格点真实面积。**全球/区域评估的正确选择。**
- **mass**：面积权 × 通量权（用参考数据的时间平均绝对值作通量权），让“通量大的区域”权重更高——适合 ET、GPP、降水等通量变量。
- **none（仅当显式写）**：简单算术平均，所有有效格点等权（高纬被高估）。

注意：反直觉：省略 `weight` → 实际用 `area`；只有显式写 `weight: none` 才是算术平均。两者结果不同——大多数用户不写此字段，即在用 `area` 加权。

9.4.3 unified_mask

默认 true。把参与对比的各模式缺测**累积**成一个共同掩码，使所有模式在完全相同的空间覆盖上被评估。这是跨模式比较公平性的关键——多模式对比时务必保持开启；只评单个模式时影响不大。

9.4.4 generate_report

默认 true，生成 HTML 报告（装 [report] 时另出 PDF）。设 false 可跳过报告、只产数据与图，适合批量/脚本场景。

9.4.5 高级开关

- debug_mode（默认 false）：在数据处理阶段打印诊断信息，排错用；不是全局日志级别。
- only_drawing（默认 false）：**跳过评估**，仅用已有结果重新绘图——改了绘图相关设置、想快速重出图时很有用。
- force（默认 false）：绕过增量缓存，**全量重算**（等同命令行 --force）。
- strict_reference（默认 false）：把”低可信”（LOW provenance）参考从警告升级为错误，强制只用可信参考。

9.5 分类分组：IGBP / PFT / Köppen

开启后，评估结果会**按地表覆盖 / 气候类别**再聚合一份：

```
project:
  IGBP_groupby: true           # 17 类 MODIS/IGBP 地表覆盖
  PFT_groupby: true           # 16 类植物功能型
  climate_zone_groupby: true  # 30 类 Köppen 气候带
```

三类掩码（IGBP.nc、PFT.nc、Climate_zone.nc）已**随包发行**（压缩约 1.1 MB），开箱即用。要用自定义/更高分辨率掩码，可用环境变量或运行目录覆盖（优先级更高）：

```
export OPENBENCH_DATASET_DIR=/path/to/your/dataset  # 最高优先
# 或在运行目录放 ./dataset/<name>.nc
```

分组结果落在 `output/<run>/comparisons/{IGBP,PFT,Climate_zone}_groupby/`。实例见第四部分案例三。

10 evaluation 与 reference: 变量与参考数据

10.1 evaluation.variables

列出要评估的**标准变量名**（非空）。变量名是 OpenBench 内部统一的”标准名”，模式/参考各自的原始变量名通过 `profile / registry` 的 `varname` 映射到它：

```
evaluation:
  variables: [Evapotranspiration, Latent_Heat, Sensible_Heat]
```

10.2 可用的标准变量名

内置参考 registry 覆盖 70+ 个标准变量名, 按领域分组如下 (用 `openbench ref show <数据集>` 可查某数据集提供哪些变量):

- **能 量/辐 射**: Latent_Heat、Sensible_Heat、Ground_Heat、Net_Radiation、Surface_Net_SW_Radiation、Surface_Net_LW_Radiation、Surface_Downward_SW_Radiation、Surface_Downward_LW_Radiation、Surface_Upward_SW_Radiation、Surface_Upward_LW_Radiation、Surface_Albedo
- **蒸散/水文**: Evapotranspiration、Transpiration、Canopy_Evaporation、Canopy_Transpiration、Bare_Soil_Evaporation、Open_Water_Evaporation、Precipitation、Runoff、Total_Runoff、Streamflow、Surface_Soil_Moisture、Root_Zone_Soil_Moisture、Soil_Moisture_Lev2、Terrestrial_Water_Storage_Change、Inundation_Area、Inundation_Fraction
- **碳/生态**: Gross_Primary_Productivity、Net_Primary_Production、Net_Ecosystem_Exchange、Ecosystem_Respiration、Biomass、Leaf_Area_Index、Burned_Area、FCH4、Methane、Wetland_Methane_Emission
- **气 象**: Surface_Air_Temperature、Diurnal_Max_Temperature、Diurnal_Min_Temperature、Surface_Soil_Temperature、Root_Zone_Soil_Temperature、Surface_Pressure、Surface_Relative_Humidity、Surface_Specific_Humidity、Surface_Wind_Speed (含 _U10/_V10) 、Cloud_Cover、Lake_Surface_Water_Temperature、Ground_Frost_Frequency
- **雪冰**: Snow_Depth、Snow_Water_Equivalent、Snow_Cover_Extent
- **城 市/人 类 活 动**: Urban_Air_Temperature_Max/_Min、Urban_Albedo、Urban_Surface_Temperature、Urban_Latent_Heat_Flux、Urban_Anthropogenic_Heat_Flux、Dam_Inflow/

Outflow、Dam_Water_Storage/_Elevation、Crop、Total_Irrigation_Amount、Water_Use

说明: 变量名大小写敏感, 须与 registry/profile 中一致。某个变量能否评估, 取决于是否有参考数据集和模式 profile 同时提供它。

10.3 reference: data_root + 变量绑定

reference 含一个公共 data_root 与”变量 → 参考源”的映射。一个变量可绑定单个或多个参考源 (列表写法); 多个时各自独立评估 (sim × ref 笛卡尔积):

```
reference:
  data_root: /data/Reference
  Evapotranspiration: ET_Xu_etal_2025_LowRes           # 单个参考
  Latent_Heat: [FLUXCOM, ILAMB_monthly]                # 多个参考, 各评一次
```

10.4 数据集 registry 与命令

参考数据集来自内置 registry (101 个)。常用命令:

```
openbench ref list           # 列出全部参考数据集
openbench ref show ET_Xu_etal_2025_LowRes  # 查看某数据集详情/变量
openbench ref scan /data/Reference  # 扫描目录自动注册已有数据
```

说明: 很多数据集有多分辨率版本 (后缀 _LowRes/_MidRes)。基名 (如 GLEAM_v4.2a) 会解析为”分辨率族”, 也可直接指定带后缀的具体名。

10.5 grid 与 station 两类参考

参考数据 data_type 分 grid (格点 NetCDF) 与 stn (站点, 逐站 NetCDF + 站点列表 CSV)。站点参考受 project.min_year_threshold 控制: 有效年限不足的站点会被剔除。注册自有数据见第五部分第 22 节。

10.5.1 站点列表 CSV 格式

站点数据 (参考或模拟) 用一份 fulllist CSV 把每个站点指向其 NetCDF。表头列为:

```
ID, SYEAR, EYEAR, LON, LAT, Flag, DIR
```

- ID: 站点标识; SYEAR/EYEAR: 该站起止年; LON/LAT: 经纬度。
- Flag: 是否参与 (真/假); DIR: 该站 NetCDF 的路径 (绝对或相对)。

可用 `openbench ref generate-station-list` 从一批站点 NetCDF 自动生成此列表 (见第五部分第 22 节)。

11 simulation 块与 model profile

11.1 条目结构

`simulation` 是” 标签 → 条目” 的字典, 可放多个被评估模型。条目字段见表 9。

表 9: `simulation.<label>` 字段 (核对自 `config.schema.SimulationEntry`)

字段	取值	含义
<code>model</code>	字符串 (必填)	模型 profile 名 (如 CoLM2024), 解析变量映射
<code>root_dir</code>	路径 (必填)	模型输出根目录
<code>data_type</code>	<code>grid stn</code>	数据形态 (覆盖 profile)
<code>grid_res / tim_res</code>	浮点 / 字符串	空间/时间分辨率 (覆盖 profile)
<code>data_groupby</code>	<code>Year Month Day single</code>	输出文件的时间切分粒度
<code>prefix / suffix</code>	字符串	文件名前缀/后缀
<code>fulllist</code>	路径	站点列表 CSV (站点模拟用)
<code>variables</code>	映射	逐变量覆盖: <code>varname</code> 、 <code>varunit</code> 、 <code>prefix</code> 等

11.2 字段取值详解

11.2.1 model

模型 profile 名 (如 CoLM2024)。匹配内置 profile 时自动套用其变量名/单位/ 文件约定 (见下小节); 不匹配也可, 只要在 `variables` 里把每个变量的 `varname` 写全。

11.2.2 root_dir

该模式输出文件所在根目录 (绝对或相对)。OpenBench 在此 (或其子目录) 按 `prefix/suffix` 查找文件。

11.2.3 data_type

grid (规则经纬网格 NetCDF) 或 stn (站点, 逐站 NetCDF + 站点列表)。决定该模式按格点还是按站点与参考配对评估。

11.2.4 data_groupby

输出文件的**时间切分粒度**, 决定 OpenBench 如何按年定位文件:

- Year: 每年一文件 (文件名含年份)。
- Month: 每月一文件 (文件名含 YYYY-MM 等)。
- Day: 每日一文件。
- single: 所有时间在一个文件里。

11.2.5 grid_res / tim_res

该模式数据的原生分辨率, 覆盖 profile 默认。最终都会被重网格/ 聚合到 project 的目标分辨率。

11.2.6 prefix / suffix

文件名前后缀; OpenBench 按 {prefix}{year}*{suffix}.nc 匹配。以点开头的后缀须加引号 (suffix: ".021")。

11.2.7 fulllist

站点模拟的站点列表 CSV 路径 (含 ID、经纬度、每站文件路径)。stn 时必需。

11.2.8 variables

逐变量覆盖映射, 每个变量可设 varname (NetCDF 内变量名)、varunit (单位, 用于换算)、以及**逐变量 prefix** (应对”按变量分文件”的模式, 见下文与案例五)。

11.3 model profile 匹配机制

当 model 匹配内置 profile (22 个) 时, 变量文件名模式与单位换算**自动套用**; 条目内字段 **覆盖** profile。查看某 profile 的变量映射: `openbench model show CoLM2024`。

11.4 文件命名: prefix / suffix / data_groupby

OpenBench 按 {prefix}{year}*{suffix}.nc 查找某年文件, data_groupby 指明切分粒度:

```
simulation:
  CoLM2024_run:
    model: CoLM2024
    root_dir: /data/CoLM2024
    data_groupby: Month          # 文件 <prefix>YYYY-MM.nc
    prefix: hist_
    variables:
      Latent_Heat: {varname: f_lfevpa, varunit: W m-2}
```

11.5 逐变量覆盖与”按变量分文件”

variables.<var> 可覆盖 varname/varunit, 也可设**逐变量 prefix**——这对”每个变量一个文件、变量名编码在文件名里”的模式(如 TE)很关键:

```
variables:
  Latent_Heat: {varname: LTNT, prefix: "YEE2_JRA-55_LTNT_M"}
  Sensible_Heat: {varname: SENS, prefix: "YEE2_JRA-55_SENS_M"}
```

完整的四模式对接实例见第四部分案例五。

注意: 以点开头的后缀(如 .021)会被 YAML 当成数字, 必须加引号写成 suffix: ".021"。

11.6 用 sim scan 自动生成 simulation 配置

手写每个模式的 prefix/suffix/data_groupby 容易出错。openbench sim scan 能扫描模式输出目录、自动推断这些字段, 并生成可直接粘进配置的 simulation 片段:

说明: sim 与 reference 的目录约定不同: 参考扫描靠固定的变量名文件夹(Grid/<分辨率>/<类别>/<变量>/<数据集>, 见第五部分第 22.4 节); 而**模拟没有变量文件夹约定**——sim scan 在 --case-depth (默认 5) 层内**发现”case”**(一个 case = 一个模式运行目录), **目录名即 case 标签**(名为 history/output 的目录则取其父目录名); 至于变量名/单位, 是由 **model profile** 映射的, 不从文件夹名推断。所以模拟数据怎么分目录都行, 关键是给对 --model。


```
# 先预览扫描到哪些 case、推断了什么（不写文件）
openbench sim scan /data/Simulation --model auto --dry-run

# 确认无误后写出 simulation 配置片段（含站点列表）
openbench sim scan /data/Simulation --model auto \
  --output sims.yaml --report scan_report.yaml -y
```

常用选项：`--model auto`（自动匹配 profile，也可指定如 `CoLM2024`）、`--case-depth N`（扫描目录深度）、`--case-pattern GLOB`/`--exclude GLOB`（筛选/排除 case）、`--climatology auto|off|year|month`、`--station-output DIR`（站点列表/合并文件输出目录）。生成的 `sims.yaml` 可用 `!include` 引入主配置（`simulation: !include sims.yaml`），再 `openbench check` 验证。

说明：`openbench init` 内部也会调用同一扫描逻辑（其 `--sim-*` 选项即对应这里，见第二部分第 6.1.2 节）。

11.7 sim scan 推不出 model profile 怎么办

`sim scan` 要为每个 case 确定“由哪个模式 profile 产出”（决定变量名/单位/文件名约定）。当某 case 无法自动匹配到已知 profile 时，会被标为 `unresolved`。这与参考扫描的“布局不识别”不同——处理办法是给它一个模式 profile：

- **指定 profile：**`sim scan ... --model CoLM2024`（强制用某个已注册 profile）；在 `openbench init` 里则会提示你输入一个 profile 名后自动重扫。
- **先注册模式：**若该模式尚无 profile，用 `openbench model register`/`model import` 添加，或 `sim scan ... --register-model NAME`（可加 `--overwrite-model`）从扫描元数据生成一份草稿 profile。
- **排除无关目录：**`--exclude GLOB`（可重复）剔除非模式输出的子目录。
- **先放行、之后再补：**`--allow-unresolved` 允许把未解析的 case 也写进 YAML（model 留空），之后手动补 model 再 `openbench check`。

完整示例。 `--model auto` 推不出 `MyRun` 这个 case 的模式时，会报错并给出处理提示：

```
$ openbench sim scan /data/Simulation --model auto --dry-run
Simulation scan reported unresolved cases:
- model inference: MyRun
```

```
Specify --model MODEL for an existing profile, create a draft with
--model NewModel --register-model, or re-run with --allow-unresolved.
For exact mappings, use `openbench model register NewModel -v StdName:
  nc_var:unit`.
Pass --allow-unresolved to publish anyway.

# A) 已有该模式的 profile —— 直接指定
$ openbench sim scan /data/Simulation --model CoLM2024 -o sims.yaml -y
Wrote sims.yaml

# B) 没 profile —— 从扫描元数据生成一份草稿 profile (名为 MyModel)
$ openbench sim scan /data/Simulation --model MyModel --register-model -o
  sims.yaml -y
Wrote draft model profile .../model_catalog.yaml
Wrote sims.yaml

# C) 先放行 —— model 留空写进 YAML, 之后手动补
$ openbench sim scan /data/Simulation --model auto --allow-unresolved -o
  sims.yaml -y
```

草稿 profile (办法 B) 写进用户 overlay 的 `model_catalog.yaml`, 可再用 `openbench model show MyModel` 查看、`model register MyModel -v Latent_Heat:LH:W m-2` 补精确的变量映射。

建议先 `--dry-run` 看哪些 case 解析成功/失败, 再决定。参考数据 (reference) 那侧的“无法识别”处理见第五部分第 22.4.1 节。

12 指标、评分、对比与统计配置

`metrics` 与 `scores` 为顶层列表; 按方法名引用。省略时分别默认 `[bias, RMSE, correlation]` 与 `[Overall_Score]`。只关心若干项时显式列出:

```
metrics: [bias, RMSE, ubRMSE, correlation, KGE, NSE]
scores:  [nBiasScore, nRMSEScore, Overall_Score]
```

12.1 指标 (metrics) 逐项说明

OpenBench 内置 25+ 指标, 按性质分组列于表 10 (s = 模拟, o = 观测/参考; 范围中 $\rightarrow$ 后为“最优值”)。

表 10: 指标逐项说明

指标	范围	含义
偏差类		
bias	$(-\infty, \infty) \rightarrow 0$	均值偏差 $\bar{s} - \bar{o}$
percent_bias	$\% \rightarrow 0$	相对偏差 $100 (\bar{s} - \bar{o}) / \bar{o}$
absolute_percent_bias	$\% \rightarrow 0$	相对偏差的绝对值
误差类		
RMSE	$[0, \infty) \rightarrow 0$	均方根误差
ubRMSE	$[0, \infty) \rightarrow 0$	去偏 RMSE = $\sqrt{\text{RMSE}^2 - \text{bias}^2}$, 只看变化幅度
CRMSD	$[0, \infty) \rightarrow 0$	中心化 RMSD (同 ubRMSE 概念, Taylor 图用)
mean_absolute_error	$[0, \infty) \rightarrow 0$	平均绝对误差
相关类		
correlation	$[-1, 1] \rightarrow 1$	Pearson 相关系数
correlation_R2	$[0, 1] \rightarrow 1$	相关系数平方 (可解释方差比例)
ubcorrelation	/ 同上	去偏版本
ubcorrelation_R2		
综合效率类		
NSE	$(-\infty, 1] \rightarrow 1$	Nash-Sutcliffe 效率 (0= 等于均值, <0 比均值还差)
KGE	$(-\infty, 1] \rightarrow 1$	Kling-Gupta 效率 (合并相关/偏差/方差比)
KGESS	$(-\infty, 1] \rightarrow 1$	KGE 技巧分 (相对参照基线)
ubNSE / ubKGE	$(-\infty, 1] \rightarrow 1$	去偏版本
index_agreement	$[0, 1] \rightarrow 1$	Willmott 一致性指数
方差 / 幅度 / 其它		
rv	$[0, \infty) \rightarrow 1$	方差比 σ_s / σ_o
kappa_coeff	$[-1, 1] \rightarrow 1$	类别一致性 (适合分类型变量)
pc_max / pc_min / pc_ampli	%	极大值/极小值/振幅的相对偏差

指标	范围	含义
rm_mean	—	去均值后的残差度量
smpi	$[0, \infty) \rightarrow 0$	单模式性能指数 (归一化均方差)
br2, cp, dr, APFB, L, MFM	—	专用/进阶指标, 公式见源码 core/metrics.py

说明: rSD、PBIAS_HF、PBIAS_LF 为预留接口, 当前显式抛 NotImplementedError, 请勿在 metrics 中使用。

12.2 评分 (scores) 逐项说明

评分把指标归一化到 $[0, 1]$ (越大越好), 便于跨变量/模式比较:

表 11: 8 个归一化评分

评分	含义
nBiasScore	基于 bias 的归一化分
nRMSEScore	基于 RMSE 的归一化分
nPhaseScore	相位一致性 (季节循环对齐程度)
nIavScore	年际变率 (interannual variability) 一致性
nSpatialScore	空间分布一致性
nSeasonalityScore	季节性一致性
index_agreement	Willmott 一致性指数 (亦可作评分)
Overall_Score	综合分 (上述加权合成), portrait/ranking 的核心输入

12.3 对比与统计配置

12.3.1 comparison: 跨模型对比图

```
comparison:
  enabled: true                # 默认 false
  items:                       # 省略 -> [Taylor_Diagram, HeatMap] (非
    全部)
    - Taylor_Diagram
    - HeatMap
    - Portrait_Plot_seasonal
    - Single_Model_Performance_Index
```

注意：省略 `items` 只出两种默认图 (Taylor + HeatMap)，**不是全部**。要出更多图种必须显式列出。

可用图种逐项说明 (表 12)：

表 12: comparison 可用图种

图种	说明
HeatMap	所有 metric/score 跨 (sim, ref) 的综合热图，一眼看全局优劣
Taylor_Diagram	极坐标图：相关系数、归一化标准差、CRMSD 三合一
Target_Diagram	(bias, ubRMSE) 散点图，看偏差与离散的组合
Portrait_Plot_seasonal	portrait 热图，每 (item, ref, sim) $\times$ 4 季节一格
Single_Model_Performance_Index	SMPI 综合得分 + bootstrap 置信, ranking 数据来源
Kernel_Density_Estimate	sim/ref 概率密度对比 (分布形状)
Parallel_Coordinates	多 metric 平行坐标图，多维一览
Whisker_Plot	多 metric 箱线图 (分布与离群)
Ridgeline_Plot	分布脊线图 (多组分布叠置)
Diff_Plot	sim - ref 偏差地图
Relative_Score	相对评分 (相对某基线)
Mann_Kendall_Trend_Test	趋势显著性检验图
Correlation	跨模型相关分析对比
Standard_Deviation	标准差对比 (变率幅度)
Functional_Response	变量间函数响应对比
RadarMap	雷达图 (多指标 $\times$ 多模型一览)

说明：`enabled: false` 时无论 `items` 写啥都不出对比产物；只有 1 个 sim 时对比图会退化为单点/单线，意义不大 (多模式对比才有价值)。Portrait 需四季 $\times$ 各 metric 都有有限数据，数据覆盖不足时会单项报错并跳过 (见第四部分案例三)。

12.3.2 statistics：独立统计分析

```

statistics:
  enabled: true                # 默认 false
  items:                       # 省略 -> 空集（即便 enabled 也不跑！）
    - ANOVA
    - Correlation
    - Mann_Kendall_Trend_Test
    - Hellinger_Distance

```

注意：与 comparison 不同：statistics.enabled: true 但省略 items → 等同没启用。要跑统计模块必须 **enabled + 显式 items**。

可用统计方法逐项说明（GUI 统计页暴露的 16 个可选项，表 13）：

表 13: statistics 可用方法

方法	说明
描述性 / 聚合	
Basic	基础统计量汇总
Mean / Median	时间维均值 / 中位数
Min / Max / Sum	时间维极小 / 极大 / 求和
Standard_Deviation	标准差（变率幅度）
Z_Score	标准化得分（异常度）
关系 / 归因	
Correlation	相关分析
Partial_Least_Squares_Regression	偏最小二乘回归（多预测因子归因）
Functional_Response	变量间的函数响应关系
趋势 / 分布 / 误差分解	
ANOVA	方差分析/分解
Mann_Kendall_Trend_Test	非参数趋势显著性检验
Hellinger_Distance	两分布相似度
False_Discovery_Rate	多重假设检验的 FDR 控制
Three_Cornered_Hat	三角帽误差分解（需 3 个数据源）

说明：Three_Cornered_Hat 需要 3 个独立数据源才有意义。底层 core/statistics/ 还有若干内部模块（如 Covariance、Variance、Autocorrelation 等）主要供程序调用，

不在上表 GUI 可选项之列。统计阶段产物落在 `output/<run>/statistics/< 方法>/。`

13 性能调优与环境变量

面向高分辨率格点或海量文件时，调 `project.num_cores` 与两个可选子块 `project.io`、`project.dask`。两个子块均可整体省略（走默认）。

13.1 io 子块：高吞吐 IO（完整字段）

表 14: `project.io` 全部字段（核对自 `config.schema.IOConfig`）

字段	默认	含义
<code>netcdf_compression</code>	<code>false</code>	数值型 NetCDF 输出是否启用 zlib 压缩
<code>netcdf_compression_level</code>	<code>1</code>	压缩级别（1= 快速默认）
<code>mfdataset_batch_size</code>	<code>None</code>	多文件读取批大小：None/auto= 资源规划器自动；0= 禁用分批；N= 每批 N 个文件
<code>mfdataset_auto_batch_min_files</code>	<code>None</code>	自动分批触发的最小文件数
<code>mfdataset_auto_batch_min_size_mb</code>	<code>None</code>	自动分批触发的最小总体积（MB）
<code>mfdataset_auto_batch_min_size</code>	<code>None</code>	自动分批的最小批尺寸（字节级阈值）
<code>mfdataset_auto_batch_max_size</code>	<code>None</code>	自动分批的最大批尺寸（上限文件数）
<code>mfdataset_auto_batch_memory_fraction</code>	<code>None</code>	分批可占用的内存比例（如 0.25）

13.2 dask 子块：分布式调度（完整字段）

13.3 配置示例

```
project:
  num_cores: 16
  io:
    netcdf_compression: true
    netcdf_compression_level: 1
    mfdataset_batch_size: auto
    mfdataset_auto_batch_min_files: 200
    mfdataset_auto_batch_memory_fraction: 0.25
```

表 15: project.dask 全部字段（核对自 config.schema.DaskConfig）

字段	默认	含义
enabled	false	是否启用 dask.distributed 调度
scheduler	None	已有调度器地址（如 tcp://...）；留空则本地起
n_workers	None	worker 数（留空自动）
threads_per_worker	1	每 worker 线程数
processes	true	用多进程（true）还是多线程（false）
memory_limit	auto	每 worker 内存上限（如 4GB、auto）
dashboard_address	None	dask 仪表盘地址（如 :8787）
local_directory	None	worker 溢写临时目录

```
dask:
  enabled: true
  n_workers: 4
  threads_per_worker: 1
  processes: true
  memory_limit: auto
  dashboard_address: ":8787"
```

说明：环境变量优先于 YAML，便于临时基准测试，例如：

```
OPENBENCH_NETCDF_COMPRESSION=1 OPENBENCH_DASK=1 OPENBENCH_DASK_N_WORKERS=4
openbench run openbench.yaml --force
```

13.4 环境变量参考

OpenBench 识别一批 OPENBENCH_* 环境变量，用于**临时覆盖**配置或指定数据/缓存位置。环境变量**优先于**YAML，适合批处理、HPC 与基准测试（改一次环境即可，不动配置文件）。

说明：此外还有少量高级/内部变量（如 OPENBENCH_ALGORITHM_VERSION、OPENBENCH_STATION_MATCHER_JOBS、OPENBENCH_ROOT_MARKERS 等），普通用户无需设置。

14 GUI 配置向导（可选）

若装了 [gui] extra，可用图形向导交互式生成配置并管理运行：

表 16: 常用环境变量（核对自源码）

变量	含义
数据 / 路径	
OPENBENCH_DATASET_DIR	分类掩码 (IGBP/PFT/Köppen) 目录, 覆盖随包版
OPENBENCH_REF_ROOT	参考数据根目录 (覆盖 registry/配置中的根)
OPENBENCH_DATA_ROOT	通用数据根目录
OPENBENCH_SIM_ROOT	模拟数据根目录
OPENBENCH_CUSTOM_DIR	自定义滤镜/扩展目录
OPENBENCH_INCLUDE_ROOTS	!include 的搜索根
IO (对应 <i>project.io</i>)	
OPENBENCH_NETCDF_COMPRESSION / _COMP_LEVEL	NetCDF 压缩开关 / 级别
OPENBENCH_MFDATASET_BATCH_SIZE	多文件读取批大小
OPENBENCH_MFDATASET_AUTO_BATCH_*	自动分批阈值 (MIN_FILES / MIN_SIZE / MIN_SIZE_MB / MAX_SIZE / MEMORY_FRACTION)
Dask (对应 <i>project.dask</i>)	
OPENBENCH_DASK	启用 dask 调度
OPENBENCH_DASK_N_WORKERS / _THREADS_PER_WORKER	worker 数 / 每 worker 线程数
OPENBENCH_DASK_MEMORY_LIMIT / _PROCESSES	内存上限 / 进程模式
OPENBENCH_DASK_SCHEDULER / _DASHBOARD_ADDRESS / _LOCAL_DIRECTORY	调度器地址 / 仪表盘 / 溢写目录
重网格权重缓存	
OPENBENCH_REGRID_WEIGHT_CACHE_DIR	重网格权重缓存目录
OPENBENCH_REGRID_WEIGHT_CACHE_MAX_SIZE / _TTL_DAYS	缓存上限 / 有效期
外部库	
HDF5_USE_FILE_LOCKING	设 FALSE 关闭 HDF5 文件锁 (NFS/共享盘)
CARTOPY_DATA_DIR	Cartopy 底图数据目录 (共享底图)

```
openbench gui          # 需先 pip install "colm-openbench[gui]"
```

14.1 界面组成

向导按**配置块**分页，与本部分各章一一对应：

- **General / Runtime**: project 的标识、时空范围、分辨率、对齐、并行、权重等。
- **Evaluation / Reference Data**: 选变量、为每个变量绑定参考数据集。
- **Simulation Data**: 登记模式输出（可调用 sim scan 扫描）。
- **Metrics / Scores / Comparisons / Statistics**: 勾选指标、评分、对比图种、统计方法。
- **Registry**: 管理参考/模型 registry（注册、查看）。
- **Preview**: 预览生成的 openbench.yaml。
- **Run Monitor**: 运行并查看进度/日志。

14.2 典型操作

1. **新建**: 逐页填写 → **Preview** 看 YAML → **Save** 导出 openbench.yaml。
2. **加载已有**: **Load Config** 打开现有 YAML，各页自动回填，可继续编辑。
3. **校验**: **Validate/Check** 等价于 openbench check，逐条提示数据/字段问题。
4. **运行**: **Run** 在 **Run Monitor** 页执行并实时显示进度；远程运行也在此配置（见第五部分第 24 节）。

GUI 导出的就是标准 openbench.yaml，与命令行**完全互通**（GUI 存的配置可用 openbench run 跑，反之亦然）。

14.3 常见问题

- openbench gui 报缺 PySide6 → pip install "colm-openbench[gui]"。
- 无图形界面的服务器（纯 SSH）无法弹窗 → 用命令行流程；GUI 适合本地工作站。
- Validate 报某变量无参考 → 同 check，按第五部分第 22.6 节排查。

第四部分 实例操作

本部分通过可复现的完整案例，演示从数据组织、编写配置、校验、运行到结果解读的全过程。案例一基于 OpenBench 随包自带的 Initial_test 样例，任何安装后的机器都能一键复现；后续案例演示扩展对比、接入自有数据、多模式横向对比与旧配置迁移。本部分案例一览：

表 17: 本部分案例一览

案例	内容	数据
一	Initial_test: 蒸散发/能量通量（格点 + 站点）全流程	内置样例（真实运行）
二	用自己的数据替换（注册参考、对接模式、站点列表）	实操指南
三	扩展对比图（Taylor）+ IGBP 分类分组	内置样例（真实运行）
四	从旧 JSON/NML 配置迁移并运行	实操指南
五	四个真实陆面模式横向对比（CoLM2024/CLM5/TE/GLDAS）	真实模式数据（真实运行）

建议阅读路径：先按案例一跑通内置样例，建立对输出与流程的直观认识；再照案例二把数据换成自己的；需要多模式互比时参考案例五。标注”真实运行”的案例，其表格数值与图件均来自实际 openbench run 输出。

15 案例一：随包自带的蒸散发 / 能量通量评估 (Initial_test)

15.1 一键复现

OpenBench 内置了一个小型评测样例 Initial_test，并提供 smoke-test 命令自动解包数据、生成配置并运行：

```
# 仅校验配置与数据（快）
openbench smoke-test

# 完整跑一遍评估，并保留工作目录以便查看产物
openbench smoke-test --run --keep
```

命令结束后会打印工作目录、配置路径，--run 还会列出关键结果产物（综合分数热图与空间分布图），例如：

```
Smoke fixture: /tmp/openbench-smoke-XXXX/Initial_test
Config:        /tmp/openbench-smoke-XXXX/openbench-smoke.yaml
...
Smoke result artifacts:
  Total score heatmap:
    - comparisons/HeatMap/scenarios_Overall_Score_comparison_heatmap.jpg
  Total score heatmap data:
    - comparisons/HeatMap/scenarios_Overall_Score_comparison.csv
  Overall score spatial maps:
    - scores/Evapotranspiration_ref_..._sim_grid_case_Overall_Score.
      jpg
    - ...
Smoke run passed.
Kept smoke work directory: /tmp/openbench-smoke-XXXX
```

smoke-test --run 会校验这些结果产物确实生成(综合分数热图 + 各对的 Overall_Score 空间图), 缺失则报错——因此它既验证”装好能跑”, 也验证”出得了结果图”。本节即以该样例为例, 逐段拆解配置、校验输出与结果产物。它同时覆盖格点与站点两种评估形态。

15.2 案例概述与数据组织

该样例评估 1 个模式的输出对多套参考数据的表现, 时段 2004–2005、月尺度、2° 格点:

- **变量:** 蒸散发 Evapotranspiration、潜热 Latent_Heat、感热 Sensible_Heat
- **模拟** (同一 CoLM 输出的两种形态): grid_case (格点) 与 station_case (约 58 个通量站点)
- **参考:** GLEAM v4.2a (格点)、GLEAM_hybrid_PLUMBER2 (站点)、ILAMB-FLUXCOM (格点)、PLUMBER2 (站点)

解包后的目录组织如下:

```
Initial_test/
+-- Reference/Initial_test/
|   +-- GLEAM4.2a_monthly/           # 格点 ET 参考 (NetCDF)
|   +-- GLEAM_hybrid_PLUMBER2/       # 站点 ET 参考 (逐站 NetCDF)
|   +-- ILAMB/Latent_Heat/FLUXCOM、ILAMB/Sensible_Heat/FLUXCOM
|   +-- PLUMBER2/                   # 站点能量通量参考 (逐站 NetCDF)
|   +-- GLEAM_hybrid_PLUMBER2.csv、PLUMBER2.csv # 站点列表
`-- Simulation/Initial_test/
```

```

+-- grid/                # 格点模拟 (grid_case_hist_*.nc)
`-- stn/                 # 站点模拟 (逐站 sim_*.nc) +
    station_case.csv

```

说明：站点参考/模拟通过**站点列表 CSV** (含 ID、经纬度、DIR 等列) 把每个站点指到对应 NetCDF; `smoke-test` 会自动把列表里的相对路径改写为本机绝对路径。

15.3 配置文件

`smoke-test` 生成的 `openbench-smoke.yaml` 即一份完整的 OpenBench 配置 (已省略自动填入的绝对路径):

```

project:
  name: openbench_initial_test_smoke
  output_dir: <work_dir>/output
  years: [2004, 2005]
  tim_res: Month
  grid_res: 2.0
  weight: none
  num_cores: 1
  time_alignment: intersection
  unified_mask: true
  generate_report: false

evaluation:
  variables: [Evapotranspiration, Latent_Heat, Sensible_Heat]

reference:
  data_root: <reference_root>
  Evapotranspiration: [OpenBench_Smoke_GLEAM4_2a,
    OpenBench_Smoke_GLEAM_hybrid_PLUMBER2]
  Latent_Heat: [OpenBench_Smoke_ILAMB_monthly,
    OpenBench_Smoke_PLUMBER2]
  Sensible_Heat: [OpenBench_Smoke_ILAMB_monthly,
    OpenBench_Smoke_PLUMBER2]

simulation:
  grid_case:                # 格点模拟
  model: CoLM
  data_type: grid
  root_dir: <sim>/grid
  prefix: grid_case_hist_
  variables:
    Evapotranspiration: {varname: f_fevpa, varunit: mm s-1}
    Latent_Heat: {varname: f_lfevpa, varunit: W m-2}
    Sensible_Heat: {varname: f_fsena, varunit: W m-2}

```

```

station_case:                                # 站点模拟
  model: CoLM
  data_type: stn
  root_dir: <sim>/stn
  fulllist: <work_dir>/lists/station_case.csv
  variables:
    Evapotranspiration: {varname: f_fevpa, varunit: mm s-1}
    Latent_Heat:         {varname: f_lfevpa, varunit: W m-2}
    Sensible_Heat:       {varname: f_fsena, varunit: W m-2}

metrics: [bias, RMSE, correlation]
scores:  [Overall_Score]
comparison: {enabled: true, items: [HeatMap]}
statistics: {enabled: false}

```

要点:

- 一个变量可绑定**多个**参考（列表写法），OpenBench 会对每个参考各评估一次。
- variables 下的 varname/varunit 把模式文件里的变量名与单位对接到标准变量；单位由 OpenBench 自动换算（如 mm s⁻¹ → mm day⁻¹）。
- 本例 weight: none（算术平均）、generate_report: false（不出 PDF/HTML，只产数据与图）。

15.4 校验: openbench check

正式运行前先校验配置与数据可用性:

```
openbench check openbench-smoke.yaml
```

真实输出（节选）:

```

Reference data (6 sources for 3 variables):
[OK] Evapotranspiration → OpenBench_Smoke_GLEAM4_2a (grid, Month)
[OK] Evapotranspiration → OpenBench_Smoke_GLEAM_hybrid_PLUMBER2 (stn,
    Day)
[OK] Latent_Heat → OpenBench_Smoke_ILAMB_monthly (grid, Month)
[OK] Latent_Heat → OpenBench_Smoke_PLUMBER2 (stn, Day)
[OK] Sensible_Heat → OpenBench_Smoke_ILAMB_monthly (grid, Month)
[OK] Sensible_Heat → OpenBench_Smoke_PLUMBER2 (stn, Day)
Evaluation tasks: 12 (6 references × 2 simulations)

Simulation data (2 models):
[OK] grid_case (model: CoLM, ...)
[OK] station_case (model: CoLM, ...)

```

```

Metrics: bias, RMSE, correlation
Scores: Overall_Score
Options: Time alignment: intersection | Unified mask: True | Comparison:
        True

[OK] Config valid (3 variables, 6 references, 2 simulations). Ready to
      run.

```

OpenBench 把 6 个 (变量, 参考) 组合 $\times$ 2 个模拟展开为 **12 个评估任务**。

15.5 运行: openbench run

```

openbench run openbench-smoke.yaml          # 增量; 改了配置只重算受影响
      部分
openbench run openbench-smoke.yaml --force  # 忽略缓存, 全量重算

```

运行依次完成预处理 (重网格/裁剪/统一掩码)、逐任务评估、对比成图。产物落在
<work_dir>/output/openbench_initial_test_smoke/:

```

output/openbench_initial_test_smoke/
+-- data/          # 预处理后的 sim/ref NetCDF (格点 + 逐站)
+-- metrics/       # 每 (变量, 参考, 模拟, metric) 一个 NC (格点) / CSV (站点) +
      图
+-- scores/        # Overall_Score 的 NC / CSV + 图
+-- comparisons/HeatMap/
|   +-- scenarios_Overall_Score_comparison.csv      # 汇总分数表
|   `-- scenarios_Overall_Score_comparison_heatmap.jpg
`-- .openbench_cache.json    # 增量缓存索引

```

15.6 结果解读

(1) **综合分数热图**。对比阶段把所有 (变量, 参考) $\times$ 模拟的 Overall_Score 汇总为一张热图 (comparisons/HeatMap/scenarios_Overall_Score_comparison.csv):

(2) **逐格点 / 逐站点产物**。标准评估为每个 (变量, 参考, 模拟) 产出三类可视化: **格点空间分布图** (逐格 NetCDF + 全球/区域地图, metrics/、scores/ 下)、**站点空间图** (站点位置按指标上色)、**站点时间序列图** (每站 obs vs sim 随时间的曲线, data/stn_.../)。站点评估还产出一行一站的 CSV:

```

scenario,ID,sim_lon,sim_lat,...,use_syear,use_eyear,KGESS,RMSE,
      correlation,bias,Overall_Score
usgs-cb-1,AT-Neu,11.32,47.12,...,2004,2005,0.902,10.73,0.969,-2.764,0.893

```

表 18: Initial_test 综合分数 (Overall_Score, 越接近 1 越好)

变量	参考	grid_case	station_case
Evapotranspiration	GLEAM4_2a	0.391	0.462
Evapotranspiration	GLEAM_hybrid_PLUMBER2	0.748	0.741
Latent_Heat	ILAMB_monthly	0.658	0.586
Latent_Heat	PLUMBER2	0.748	0.772
Sensible_Heat	ILAMB_monthly	0.645	0.466
Sensible_Heat	PLUMBER2	0.704	0.718

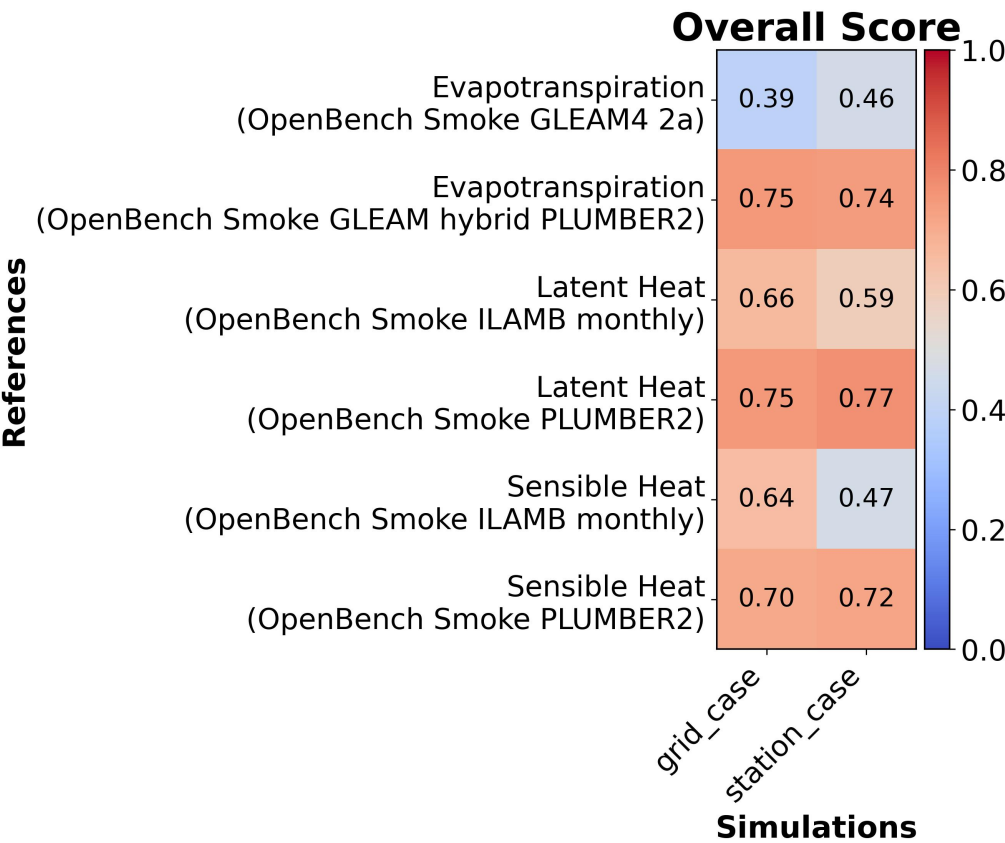


图 3: 综合分数热图 (scenarios_Overall_Score_comparison_heatmap.jpg)。

15 案例一：随包自带的蒸散发 / 能量通量评估 (Initial_test)

```
usgs-cb-1,AU-How  
,131.15,-12.50,...,2004,2005,0.700,34.82,0.882,-26.62,0.652
```

(上为 scores/Latent_Heat_stn_OpenBench_Smoke_PLUMBER2_station_case_evaluations.csv 节选。)

格点空间分布图 (图 4) 一眼看出指标的空间格局——哪里好、哪里偏差大：

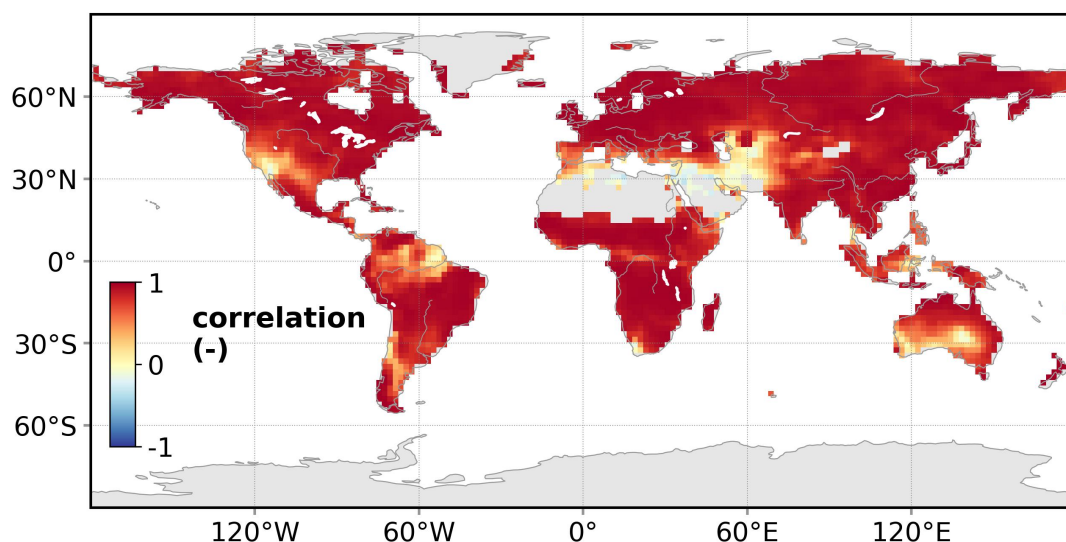
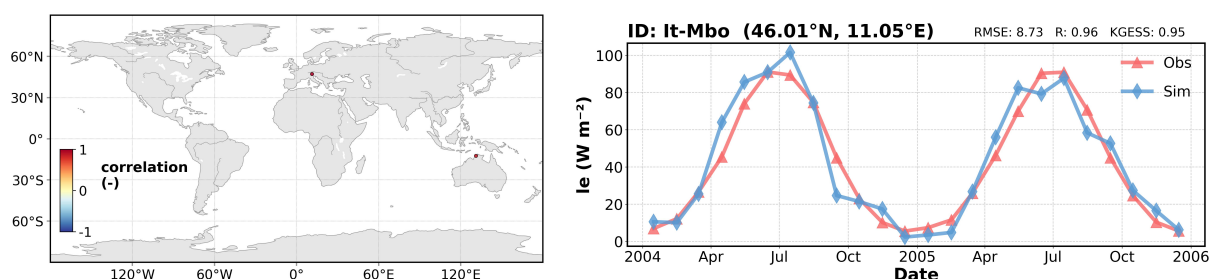


图 4: 格点空间分布图示例：潜热 correlation (CoLM grid_case 对 ILAMB-FLUXCOM)。植被区普遍高相关 (红)，干旱/过渡带偏低；灰色为掩码/无数据。

站点结果有两种视图 (图 5)：站点空间图看”哪些站点好”，站点时间序列图看单站 obs 与 sim 随时间是否吻合：



(a) 站点空间图：各站潜热相关系数

(b) 站点时间序列：IT-MBo 站 obs vs sim (R=0.96)

图 5: 站点结果两视图：左为站点空间分布图，右为单站时间序列图 (data/stn_.../<var>__<ID>__timeseries.jpg, 红 = 观测、蓝 = 模拟)。

(3) 读法小结。同一变量对不同参考的分数差异，往往反映参考数据本身的不确定性 (如 ET 对 GLEAM4_2a 偏低、对 GLEAM_hybrid_PLUMBER2 偏高)；grid 与 stn 两列的差异则反映评估形态 (区域格点 vs 站点尺度) 的影响。要复算或二次分析，直接用

xarray/pandas 读 metrics/、scores/ 下的 NC/CSV 即可（见第五部分第 25 节）。

注意：完整运行在干净机器上绘图时可能触发 Cartopy 下载 Natural Earth 底图；离线环境请先按第 5.2.2 节手动放好底图。

16 案例二：用自己的数据替换

本案例以案例一为骨架，演示把参考与模拟换成自有数据。分四步：组织数据、注册参考、对接模型、改配置。

16.1 第 1 步：组织数据目录

建议格点与站点分开放，站点数据每站一个 NetCDF：

```
MyData/
+-- Reference/MyET/          # 参考：格点 NetCDF 或逐站 NetCDF
`-- Simulation/MyModel/      # 模拟：模式输出 NetCDF
```

16.2 第 2 步：注册参考数据集

两种方式（写入用户 overlay，不动包内置 catalog）：

```
# 方式 A：扫描目录自动识别
openbench ref scan /data/MyData/Reference

# 方式 B：手工注册（指定根目录、变量名、单位等）
openbench ref register MyET --root-dir /data/MyData/Reference/MyET
openbench ref show MyET          # 确认变量映射与路径
```

站点参考还需一份站点列表 CSV（含 ID、经纬度、DIR 指向每站 NetCDF）：

```
openbench ref generate-station-list /data/MyData/Reference/MyET \
-o /data/MyData/MyET_stations.csv
```

16.3 第 3 步：对接模型

若你的模式不在内置 22 个 profile 内，注册一个，或直接在配置里用 variables 写明变量名/单位：

```
openbench model show CoLM2024      # 参考已有 profile 的写法
openbench model register MyModel    # 或注册自有 profile
```

16.4 第4步：改配置并运行

把案例一的配置里 `reference` 与 `simulation` 换成自己的名字与路径：

```
evaluation:
  variables: [Evapotranspiration]
reference:
  data_root: /data/MyData/Reference
  Evapotranspiration: MyET
simulation:
  my_run:
    model: MyModel          # 或内置 profile 名
    root_dir: /data/MyData/Simulation/MyModel
    data_type: grid
    variables:
      Evapotranspiration: {varname: ET, varunit: mm day-1}
```

```
openbench check my.yaml          # 务必先校验：数据可达、变量可解析
openbench run    my.yaml
```

说明：先 `check` 再 `run`：`check` 会逐条报告每个 (变量, 参考) 是否定位成功、共展开多少评估任务，能在跑之前发现路径/变量名/单位的问题。

17 案例三：扩展对比图与分类分组

本案例在案例一的基础上，**只改配置**就开启更多对比图（Taylor、Portrait）与 IGBP 分类分组，演示对比阶段的丰富产物。仍用内置样例即可复现。

17.1 配置改动

在案例一的 `openbench-smoke.yaml` 上做两处修改：

```
project:
  IGBP_groupby: true          # 开启 IGBP 地表覆盖分类分组

comparison:
  enabled: true
```

```
items: [HeatMap, Taylor_Diagram, Portrait_Plot_seasonal] # 增加两种图
```

然后照常运行（`openbench run openbench-smoke.yaml --force`）。新增产物落在 `comparisons/Taylor_Diagram/`、`comparisons/Portrait_Plot_seasonal/`、`comparisons/IGBP_groupby/`。

17.2 Taylor 图

Taylor 图把每个 (sim, ref) 的相关系数、归一化标准差、CRMSD 画在同一极坐标里，一眼看出“谁更接近参考”。配套 CSV 给出统计值，例如 Latent_Heat 对 PLUMBER2 的 station_case：标准差 40.37、相关 0.925、参考标准差 37.33；CRMSD 有两列——图上用反推值 15.30（由 std/corr 按 Taylor 恒等式得，保证点落在等值线上），CSV 另记真实平均 16.41（详见案例五的 CRMSD 两列说明）。

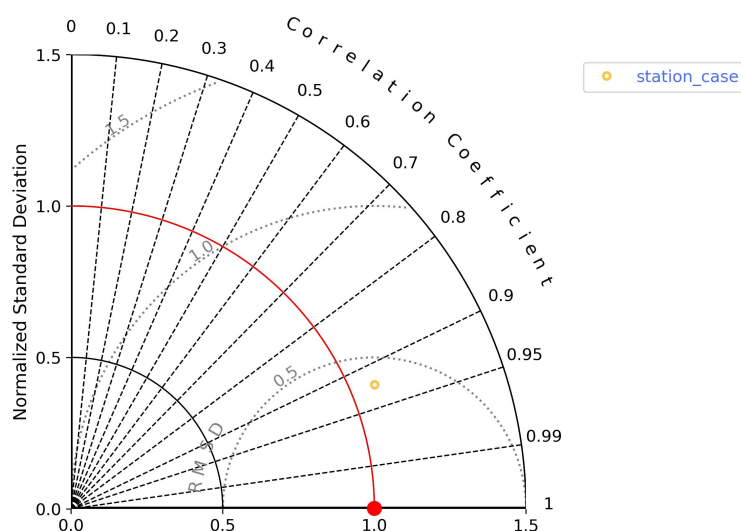


图 6: Latent_Heat 对 PLUMBER2 的 Taylor 图(Taylor_Diagram_Latent_Heat_..._PLUMBER2.jpg)。

17.3 IGBP 分类分组

开启 IGBP_groupby 后，OpenBench 用随包自带的 IGBP 掩码把格点结果按 17 类地表覆盖再聚合一份，输出逐类 CSV/热图(`comparisons/IGBP_groupby/<sim>__<ref>/`)。下表为格点蒸散发对 GLEAM4_2a 的逐类指标（节选真实输出，N/A 表示该类在本样例域内无有效格点）：

表 19: ET (grid_case vs GLEAM4_2a) 按 IGBP 类的指标 (节选)

IGBP 类	bias	RMSE	correlation
evergreen_needleleaf_forest	-33.17	44.02	0.954
deciduous_broadleaf_forest	-47.25	57.34	0.932
mixed_forests	-36.10	48.43	0.956
grasslands	-29.94	36.13	0.888
croplands	-40.54	48.78	0.888
Overall	-18.06	24.92	0.911

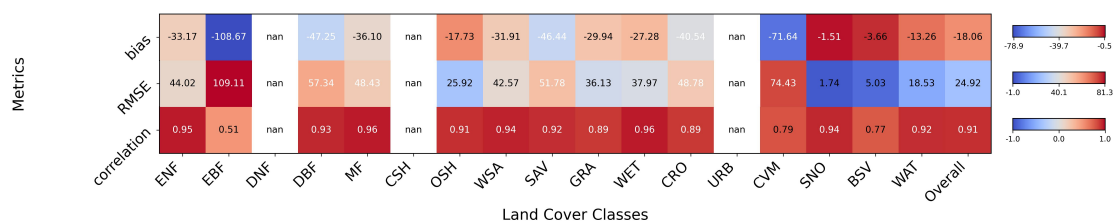


图 7: ET 按 IGBP 类的指标热图 (..._metrics_heatmap.jpg)。

说明：逐类分组能定位”模型在哪种地表覆盖上偏差最大”。本样例为小型演示数据，数值仅示意结构，不代表真实模式性能。

17.4 关于 Portrait 与”带错误完成”

Portrait (季节 portrait) 需要每个 (变量, 参考, 模拟) 在四季 × 各 metric 上都有有限数据。本内置样例数据量很小, station_case 在某些季节缺有限值, 因此 Portrait 这一项会报错并跳过:

```
[X] Evaluation completed with errors
- [comparison] Portrait_Plot_seasonal comparison failed:
  no finite data for Evapotranspiration/.../station_case/bias_DJF
```

这是数据覆盖不足所致, 并非缺陷: OpenBench 会把失败的子项单独报错, 其余图种 (HeatMap、Taylor、IGBP 分组) 照常产出; 命令以非零码退出以提示”有子项未完成”。在覆盖完整的真实数据上, Portrait 会正常出图。

18 案例四：从旧配置迁移并运行

若你已有 v2.x 的旧配置 (JSON 或 Fortran NML), 用 openbench migrate 转成新的 openbench.yaml:

```
# JSON 旧配置
openbench migrate old_config.json -o openbench.yaml

# Fortran NML 旧配置 (需要 [migration] extra: pip install "colm-openbench
[migration]")
openbench migrate main.nml -o openbench.yaml
```

迁移把旧的多文件 / NML 字段映射为新的八大块结构 (project / evaluation / reference / simulation / ...)。转换完成后，**务必先校验再运行**：

```
openbench check openbench.yaml      # 核对迁移结果：变量、参考绑定、路径是
    否完整
openbench run    openbench.yaml
```

说明：迁移后建议人工通读一遍 openbench.yaml：重点检查变量名大小写、每个变量的参考是否都成功绑定、root_dir 路径是否需要按新机器调整。OpenBench 3.0 的评估结果与 2.x 数值一致。

19 案例五：多个真实陆面模式的横向对比

本案例是 OpenBench 最核心的用法：把多个不同陆面模式放在同一把尺子下对比。我们评估 CoLM2024、CLM5、TE、GLDAS 四个模式的潜热 (Latent_Heat) 与感热 (Sensible_Heat)，统一对参考 ILAMB-FLUXCOM、时段 2004–2005、重网格到 2°。本节所有数值均为真实运行输出。

19.1 数据与难点：四个模式的命名各不相同

四个模式原生分辨率、文件命名、变量名都不同，这正是配置时要逐模式对接的地方：

表 20: 四个模式的关键差异 (实测)

模式	原生分辨率	文件组织	LH 变量名	SH 变量名
CoLM2024	~2°	逐月一文件	f_lfevpa	f_fsena
CLM5	~2°	逐月一文件 (h0 流)	EFLX_LH_TOT	FSH
TE	0.5°	逐变量逐年一文件	LTNT	SENS
GLDAS	0.25°	逐月一文件 (NOAH)	Qle_tavg	Qh_tavg

19.2 配置：逐模式对接文件约定

每个 simulation 条目用 prefix/suffix/data_groupby 描述该模式的文件命名，用 variables 对接变量名与单位。OpenBench 按 {prefix}{year}*{suffix}.nc 查找文件：

```
simulation:
  CoLM2024:
    model: CoLM2024
    root_dir: /data/LSMs/CoLM2024
    data_type: grid
    data_groupby: Month
    prefix: Global_Grid_2x2_..._hist_      # 文件 <prefix>YYYY-MM.nc
    variables:
      Latent_Heat: {varname: f_lfevpa, varunit: W m-2}
      Sensible_Heat: {varname: f_fsena, varunit: W m-2}
  CLM5:
    model: CLM5
    root_dir: /data/LSMs/CLM5
    data_groupby: Month
    prefix: IHistCln50BgcCropGSWP_CWD-1901-2014.cln2.h0.  # 只取 h0 流
    variables:
      Latent_Heat: {varname: EFLX_LH_TOT, varunit: W m-2}
      Sensible_Heat: {varname: FSH, varunit: W m-2}
  TE:
    # 逐变量分文件 -> 把 token 写进逐变量 prefix
    model: TE
    root_dir: /data/LSMs/TE
    data_groupby: Year
    suffix: _GLB050
    variables:
      Latent_Heat: {varname: LTNT, varunit: W m-2, prefix: "YEE2_JRA-55_LTNT_M"}
      Sensible_Heat: {varname: SENS, varunit: W m-2, prefix: "YEE2_JRA-55_SENS_M"}
  GLDAS:
    model: GLDAS
    root_dir: /data/LSMs/GLDAS
    data_groupby: Month
    prefix: GLDAS_NOAH025_M.A      # 文件 ...A YYYYMM .021.nc
    suffix: ".021"                # 注意：以点开头的后缀必须加引号
    variables:
      Latent_Heat: {varname: Qle_tavg, varunit: W m-2}
      Sensible_Heat: {varname: Qh_tavg, varunit: W m-2}
```

注意：两个实测踩坑：(1) TE 把变量名编码在文件名里 → 用逐变量 prefix (variables.<var>.prefix) 把 token 带进去；(2) GLDAS 后缀 .021 会被 YAML

当成数字 0.021，必须写成 ".021"。

19.3 校验与运行

```
openbench check multi.yaml          # 展开 8 个任务 (2 变量 × 4 模式)
openbench run    multi.yaml --force
```

OpenBench 自动把 0.25° (GLDAS)、0.5° (TE)、2° (CoLM2024/CLM5) 统一重网格到目标 2° 再逐对评估，无需手工预处理。

19.4 结果：综合分数热图

对比阶段把四个模式的 Overall_Score 汇总成一张表（真实输出）：

表 21: 四模式综合分数（对 ILAMB-FLUXCOM，越接近 1 越好）

变量	CoLM2024	CLM5	TE	GLDAS
Latent_Heat	0.724	0.712	0.697	0.693
Sensible_Heat	0.687	0.694	0.610	0.566

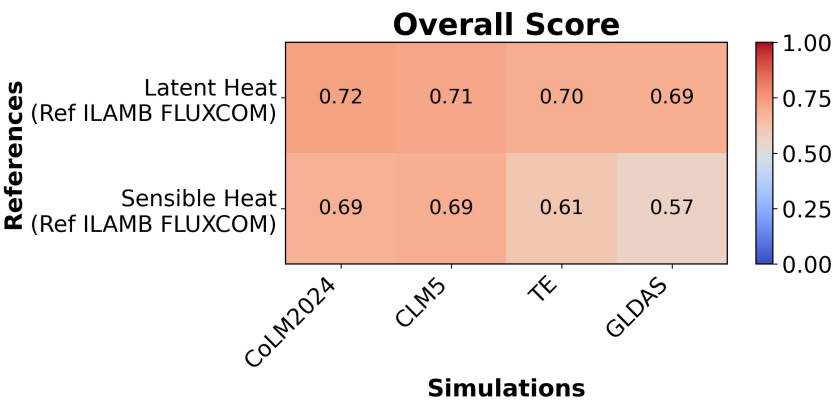


图 8: 四模式综合分数热图。

19.5 结果：Taylor 图

Taylor 图把四个模式的相关系数、标准差、CRMSD 画在一张图上对比（潜热，真实数值）。下表列原始标准差；图中是归一化标准差（= 标准差 ÷ 参考标准差），即点到原点的径向距离（参考点在径向 1.0、相关 1.0 处）。例如 CoLM2024 径向 = 20.13/24.89 = 0.81。

表 22: 潜热 Taylor 统计 (参考标准差 = 24.89 W m^{-2})

模式	标准差	相关系数	CRMSD (图)	CRMSD (真实平均)
CoLM2024	20.13	0.898	11.17	9.49
CLM5	14.91	0.855	14.41	11.01
TE	21.97	0.887	11.50	10.65
GLDAS	22.57	0.864	12.60	10.58

说明：表中两列 CRMSD 含义不同：**CRMSD (图)** 由聚合后的标准差与相关系数按 Taylor 恒等式 $\text{CRMSD} = \sqrt{\sigma_s^2 + \sigma_r^2 - 2\sigma_s\sigma_r r}$ 反推，是图上点到参考点的实际距离，保证点严格落在 CRMSD 等值线上；**CRMSD (真实平均)** 是各格点/站点 CRMSD 的直接平均。二者不等是因为“逐点统计量各自平均”不满足 Taylor 恒等式——OpenBench 因此用前者画图、用后者记录真实误差量 (CSV 中分别为 `<sim>_RMS` 与 `<sim>_RMS_mean` 两列)。

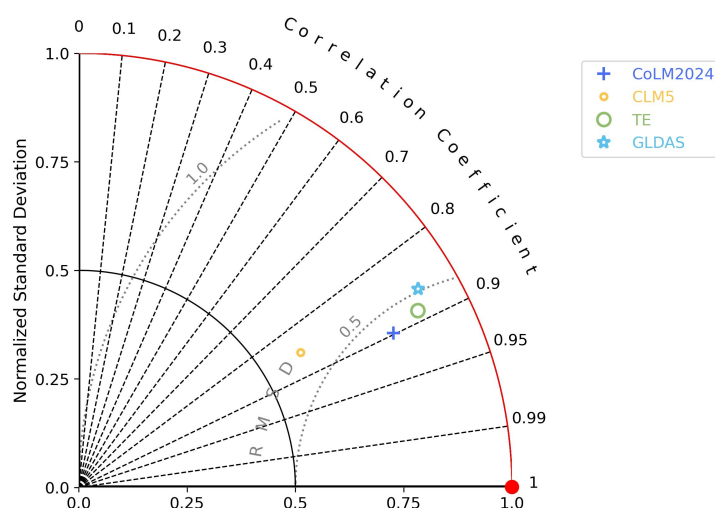


图 9: 潜热 Taylor 图 (四模式对 ILAMB-FLUXCOM)。

19.6 读法小结

综合 HeatMap 与 Taylor：潜热上 CoLM2024 相关最高 (0.898)、图上 CRMSD 最小 (11.17)，综合分领先；CLM5 标准差偏低 (变率偏弱) 但偏差小；感热整体比潜热难评 (各模式分数更低，GLDAS 最低)。这类**横向对比**正是 OpenBench 的价值：一份配置、统一口径，直接回答“哪个模式在哪个通量上更好”。

说明：要进一步定位差异来源，可加 `IGBP_groupby` (按地表覆盖看分类表现，见案例三) 或更多对比图种 (Portrait、Parallel_Coordinates 等，见第三部分第 12.3 节)。

20 结果是否合理：读图指南

20.1 快速定位结果

先按“我想看什么”定位文件（路径相对 output/<project_name>/）：

我想看…	去哪
总分（综合分）	comparisons/HeatMap/scenarios_Overall_Score_comparison_heatmap.jpg (数据 ..._comparison.csv)
格点空间分布	scores/<var>..._Overall_Score.jpg、 metrics/<var>..._<metric>.jpg（全球/区域地图）
站点空间图	metrics/<var>_stn_<R>_<S>_<metric>.jpg（站点按指标上色）
站点时间序列图	data/stn_<R>_<S>/<var>__<ID>__timeseries.jpg（单站 obs vs sim 曲线）
站点逐站指标	scores/<var>_stn_<R>_<S>_evaluations.csv(metrics/ 下亦有， 一行一站)
逐格点 metric 数据	metrics/<var>_ref_<R>_sim_<S>_<metric>.nc
分类分组结果	comparisons/{IGBP,PFT,Climate_zone}_groupby/
对比图（Taylor 等）	comparisons/< 图种 >/
调试运行失败	run.log（按阶段记录每个任务的成功/失败与原因）

20.2 怎么判断好坏

跑出结果后，如何快速判断好坏与发现异常：

20.2.1 评分（scores）

归一化到 [0, 1]，越接近 1 越好。Overall_Score 是综合分，是多模式排名与 portrait 的核心；单看某方面用 nBiasScore（偏差）/nRMSEScore（误差）等。经验上 > 0.7 较好、0.4–0.7 一般、< 0.4 偏差较大（但绝对阈值随变量/参考而异，重在横向对比）。

20.2.2 指标（metrics）方向

bias 看正负与量级（0 最好，正 = 高估）；RMSE/ubRMSE 越小越好；correlation 越接近 1 越好（时间/空间吻合）；KGE/NSE ≤ 1 ，越大越好，< 0 说明还不如用观测均值。

20.2.3 Taylor 图

一个点 = 一个模式。**越靠近参考点 (REF) 越好**：径向距原点的距离是标准差（与参考等长说明变率幅度对）、方位角是相关系数（越靠横轴相关越高）、到参考点的距离是 CRMSD（越小越好）。多模式对比时，“最靠近 REF 的点”即最优。

20.2.4 HeatMap

行列为 (变量/参考) × 模式，颜色深浅是分数。**空白/NaN** 表示该组合没有结果——通常是数据不交叠、变量缺失或该对评估失败（去 `run.log` 查原因），不是“分数为 0”。

20.2.5 格点空间图

看偏差的**空间格局**：是否某区域系统性偏高/偏低、海陆边界/高纬是否异常、是否大片 NaN（掩码或数据缺测）。结合 `IGBP_groupby` 可定位“哪种地表覆盖上最差”。

20.2.6 站点散点 / CSV

站点评估每行一站，`correlation/KGESS` 等逐站给出；个别站点极端值常是该站观测质量问题，可结合 `min_year_threshold` 或自定义滤镜剔除。

注意：“看起来很差”先排除**口径问题**再下结论：单位是否换算正确（`varunit`）、时间/空间是否对齐、`weight` 是否符合预期、是否在用气候态模式。这些比“模式真的差”更常见。

第五部分 研发用户指南

本部分面向需要**扩展** OpenBench、**接入**自有数据/模型、**二次分析**或**参与开发**的研发用户。更深入的内部实现细节请直接查阅源码 `src/openbench/`。

21 扩展点总览

OpenBench 的源码组织为若干子包，扩展点集中在 `data`（数据/registry）、`config`（配置/迁移）、`core`（指标/评分/统计/可视化桥）与 `runner`（编排/缓存）。

表 23: `src/openbench/` 子包与主要职责

子包	职责 / 常见扩展点
<code>cli</code>	命令行入口(<code>run/check/ref/model/sim/cache/migrate/gui/smoke-test</code>)
<code>config</code>	配置加载、schema、迁移、legacy 处理
<code>core</code>	评估引擎: <code>metrics</code> 、 <code>scores</code> 、 <code>comparison</code> 、17 个 <code>statistics</code> 、可视化桥
<code>data</code>	数据处理、registry（参考/模型目录）、缓存、自定义滤镜
<code>runner</code>	任务规划、并行执行、缓存指纹、断点续算、后处理
<code>visualization</code>	各类图件 (<code>Fig_*</code>) 与色表
<code>remote</code>	SSH 远程执行
<code>gui</code>	可选图形向导

表 24: 常见任务 → 去哪里改

我想…	去
加 格点 参考数据	第 22 节”格点参考数据集（条目示例）”
加 站点 参考数据	第 22 节”站点参考数据集（两种形态）”
加 新模式 (profile)	第 22.8、22.8.1 节（结构示例）
端到端走一遍接入	第 22.6 节（7 步验收）/ 第四部分案例二
扫描模拟输出生成配置	<code>openbench sim scan</code> （第三部分第 11.6 节）
新增评价指标/评分	<code>core/metrics.py</code> 、 <code>core/scores.py</code> （第 23 节）
新增统计/图件	<code>core/statistics/</code> 、 <code>visualization/</code> （第 23.2 节）
对参考做质控筛选	<code>data/custom/</code> 滤镜（第 23.3 节）
二次分析结果	<code>xarray/pandas</code> 读 NC/CSV（第 25 节）

22 注册参考数据与模式 profile

把自有参考数据接入 registry 有两条路：扫描自动注册与手工注册。

```
# 1) 扫描目录，自动识别并注册可用数据集
openbench ref scan /data/MyReference

# 2) 手工注册/更新单个数据集
openbench ref register MyET --root-dir /data/MyReference/MyET

# 3) 注册"参考 profile"（变量名/单位映射模板，供多数数据集复用）
openbench ref register-profile ...

# 查看与定位
openbench ref show MyET
openbench ref path MyET
```

注册写入用户 **overlay**（位于用户主目录的 `.openbench/` 下），不改动包内置 catalog，便于个人/团队扩展。

22.1 参考数据集的关键字段

一个参考条目需声明：`data_type` (`grid/stn`)、`tim_res`、`grid_res`、`years`、`root_dir`，以及每个变量的 `varname` (NetCDF 内变量名)、`varunit` (用于单位换算)、`prefix/suffix/sub_dir` (文件定位)。站点数据集还需**站点列表 CSV** (含 ID、经纬度、DIR 指向每站 NetCDF)，可用 `openbench ref generate-station-list` 自动生成。完整的参考 overlay 写法见第四部分案例五（其 ILAMB/GLEAM 参考即以 overlay 形式接入）。

22.2 格点参考数据集（条目示例）

格点参考按 `data_type: grid` 声明，关键是 `grid_res` 与每变量的文件定位 (`sub_dir/prefix/suffix`):

```
MyGridET:
  name: MyGridET
  category: Water
  data_type: grid
  tim_res: Month
  grid_res: 0.5
  years: [2001, 2010]
```

```

root_dir: /data/MyReference          # 也可用 ${OPENBENCH_REF_ROOT}/...
variables:
  Evapotranspiration:
    varname: et                      # NetCDF 内变量名
    varunit: mm month-1             # 原始单位（自动换算到评估单位）
    sub_dir: ET/monthly             # root_dir 下子目录
    prefix: ET_                     # 文件 <prefix><year><suffix>.nc
    suffix: _v1

```

22.3 站点参考数据集（两种形态）

站点参考有**两种**组织方式，按数据形态择一：

(a) 逐站一个 NetCDF + 站点列表 CSV（如 PLUMBER2/FLUXNET 风格）：

```

MyStnLH:
  name: MyStnLH
  category: Energy
  data_type: stn
  tim_res: Day
  data_groupby: single
  years: [2004, 2005]
  root_dir: /data/MyReference/MyStnLH
  fulllist: /data/MyReference/MyStnLH/stations.csv # 站点列表
  variables:
    Latent_Heat: {varname: Qle, varunit: W m-2}

```

站点列表 CSV 列为 ID,SYEAR,EYEAR,LON,LAT,Flag,DIR（DIR 指向每站 NetCDF），可用 `openbench ref generate-station-list < 目录 > -o stations.csv` 自动生成。

(b) 单个合并 NetCDF（含 station 维）+ station_matching（如河流站点合并文件）：

```

MyStreamflow:
  name: MyStreamflow
  category: Water
  data_type: stn
  tim_res: Month
  root_dir: /data/MyReference/Streamflow
  station_matching:
    dataset_file: streamflow_all.nc # 单一合并文件
    station_id_var: station         # 站点维/ID 变量
    lat_var: Lat
    lon_var: Lon
    time_var: time
    discharge_var: Disch           # 目标变量
    area_var: upstream_area        # 流域面积（径流匹配用）
    method: cama_allocation        # 站点-河网匹配方法

```

22.4 用 ref scan 批量发现并注册

数据多时，逐个 ref register 很累。openbench ref scan 能按约定目录结构自动发现并注册一批参考数据集。它期望的目录布局是：

```
# 格点：单变量数据集（每个数据集只含一个变量）
<REF_ROOT>/Grid/{LowRes,MidRes,HigRes}/<category>/<variable>/<dataset>/*.nc
# 格点：复合数据集（一个 NetCDF 含多个变量）
<REF_ROOT>/Grid/{LowRes,MidRes,HigRes}/Composite/<dataset>/*.nc
# 站点
<REF_ROOT>/Station/<category>/<variable>/<dataset>/*.nc
```

每一层文件夹名都有含义，扫描器据此推断元数据：

- Grid / Station：数据类型（data_type: grid / stn）。
- LowRes / MidRes / HigRes：分辨率档（高/中/低）。同一数据集跨档会按基名归组，命名为 <dataset>_LowRes 等，GUI 据此提供分辨率选择。
- <category>：类别（如 Water、Energy、Carbon）。
- <variable>：标准变量名——这层文件夹名就是该数据集提供的变量，如 Surface_Albedo、Latent_Heat、Evapotranspiration（须与 evaluation.variables 用的名字一致）。
- Composite：特殊”类别”占位——当一个 NetCDF 同时含多个变量时放这里（不再分单变量文件夹），扫描器从文件内变量推断映射。
- <dataset>：数据集名（如 GLEAM_v4.2a）。

推荐”先预览、再注册”：

```
openbench ref scan /data/Reference --dry-run      # 只看会注册什么，不写
openbench ref scan /data/Reference --auto         # 确认无误后批量注册（-y）
openbench ref scan /data/Reference --rescan        # 已注册的也刷新
openbench ref scan /data/Reference --only "GLEAM*" # 只处理匹配名的
```

扫描结果写入用户 **overlay**（主目录 .openbench/ 下的 catalog），随后用 ref list / ref show 即可看到。

22.4.1 扫描遇到无法识别的数据怎么办

当某个文件夹**不符合约定布局**时（常见原因：NC 内变量歧义/多个候选、NC 文件嵌套过深、目录结构不被识别），扫描会把它列为 `skipped`（打印路径与原因），并在交互式扫描（含 `openbench init`）里让你**四选一**：

- [s] **暂时跳过**：本次不纳入这些文件夹，其余照常注册；下次扫描仍会再问。
- [p] **建/改 profile 后重扫**：交互式为该数据定义一份**参考 profile**（指定 layout 与变量名/单位），让 OpenBench 学会怎么读它，然后自动重扫并注册。适合”数据有用、只是布局非标准”。
- [i] **加忽略 profile 后重扫**：把该文件夹**永久标记为忽略**（写一条 `layout: ignore profile`），以后扫描不再提示。适合”无关 / 非数据”的文件夹。
- [a] **中止**：取消本次扫描。

完整示例。 假设 `/data/MyRef/Grid/MidRes/Water/Evapotranspiration/MyET/` 下放多个 NetCDF 子目录,扫描器无法确定哪个才是数据集(原因 `ambiguous_nc_subdirectories`):

```
$ openbench ref scan /data/MyRef
Scanning /data/MyRef (dry run)...
[DRY RUN] Found 8 dataset(s) to register/update. No catalog changes made yet.
- Grid/MidRes/Water/Evapotranspiration/MyET:
    ambiguous_nc_subdirectories
Skip 1 unsupported folder(s), add a profile, ignore them, or abort:
[s] skip for now
[p] create/update reference profile and rescan
[i] add ignore profile and rescan
[a] abort
Action [s]: p                # 选 p: 为它建一个参考 profile
Create reference profile for Grid/MidRes/Water/Evapotranspiration/MyET
Profile name [MyET]: MyET    # 回车用默认名
# 接着按提示填: layout (数据布局)、文件 glob、每个变量的 varname/
varunit
Updated reference profile: MyET
Rescanning...                # 自动重扫
[DRY RUN] Found 9 dataset(s) to register/update.    # MyET 现在能识别了
```

选 [p] 会把新 profile 写进用户 overlay (`~/.openbench/.../reference_profiles.yaml`) 并自动重扫，之后 MyET 即可正常注册；若该文件夹本就无关（如存放说明文档），改选 [i]，它会写一条 `layout: ignore` 让以后扫描不再提示。

若某文件夹连 **profile** 都无法推断，OpenBench 会提示改用 [i] 忽略、[s] 跳过，或**手动注册**：`openbench ref register NAME --root-dir ...`（直接给全字段），或先 `openbench ref register-profile` 建模板再套用。

说明：非交互/批量(CI)场景：加 `--allow-skip` 让扫描自动跳过无法识别的文件夹、不阻塞，其余可识别的照常注册；想先看会跳过哪些而不写任何改动，用 `--dry-run`。

说明：以上 [s]/[p]/[i]/[a] 与 `--allow-skip` 针对的是**参考扫描的**”布局/变量不识别”。扫描**模式**（`openbench sim scan ... --model auto`）的”无法识别”是另一回事——推不出 *model profile*，处理办法（`--model` 指定 / `--register-model` 生成草稿 / `--allow-unresolved` 先放行）见第三部分第 11.7 节。

22.5 追加、移动或删除参考根目录

追加新的参考根（不影响已注册的）。registry 是**累积合并**的：再扫一个新根、或单独注册一个数据集，都会并入现有用户 catalog，旧条目原样保留——OpenBench 专门防止”扫新根把旧的覆盖删掉”。

```
openbench ref scan /new/ref/root          # 把新根里的数据集并入现有
registry
openbench ref register MyDS --root-dir /elsewhere/MyDS # 单个数据集，任
意路径
```

每个数据集各记自己的 `root_dir`，可来自不同路径；因此一份配置能同时用多个根下的参考。

说明：可移植性：在 `OPENBENCH_REF_ROOT` 之下扫到的存为 `${OPENBENCH_REF_ROOT}/...`（换机器只改环境变量）；在它之外的存**绝对路径**（照用，但换机器需改）。建议把主参考库设为 `OPENBENCH_REF_ROOT`，零散数据集用绝对路径补。

数据移动了。把数据集重新注册到新路径即可覆盖旧条目：`openbench ref register NAME --root-dir /new/path`（或 `ref scan` 新根）。

原路径已删除（条目失效）。若某参考的数据目录被删了，registry 里的条目就成了**悬空指向**：`openbench check` 会把它标为 [X]（数据不可用），`openbench ref path NAME` 可看它指向的路径以核对。清理办法：

```
openbench ref path MyDS          # 看它指向哪（确认确实没了）
openbench ref delete MyDS        # 从用户 catalog 删除该条目（自动备份）
```

注意： `ref delete` 只删用户 **overlay** 里的条目，删不了**内置**参考。内置参考的数据来自 `OPENBENCH_REF_ROOT`——若是那批数据被移走，**不要**删条目，改为把 `OPENBENCH_REF_ROOT` 指到数据现在的位置即可。`ref scan/rescan` 只增量添加/更新、不会自动删除已不存在的条目；要批量清理失效条目，逐个 `ref delete`，或直接编辑 `~/.openbench/references/reference_catalog.yaml`（删前会自动备份）。

22.6 端到端接入与验收流程

按以下步骤接入自有参考，并**逐步验收**，可避免”注册了但 `check` 找不到数据”：

1. **准备目录：**把数据按 `<root>/<sub_dir>/<prefix><year><suffix>.nc` 组织（站点则每站一个 NC）。
2. **注册：**`openbench ref scan /data/MyReference`（自动）或 `openbench ref register MyET --root-dir ...`（手工）。
3. **验收注册：**`openbench ref show MyET` 确认变量映射/分辨率/年份无误；`openbench ref path MyET` 确认解析到的实际路径存在。这一步能看到的，*check/run* 才找得到。
4. **(站点)生成列表：**`openbench ref generate-station-list /data/MyReference/MyET -o stations.csv`。
5. **写入 YAML：**在 `reference` 里把变量绑定到 `MyET`（`data_root` 指向其根）。
6. **校验：**`openbench check my.yaml`——逐条看每个（变量, 参考）是否 [OK]。
7. **最小运行：**先用 `--variable` 只跑一个变量验证全链路：`openbench run my.yaml --variable Evapotranspiration`。

注意：”注册了但 `check` 找不到数据”的常见原因：(1) 变量名大小写/拼写与 `evaluation.variables` 不一致；(2) `prefix/suffix/sub_dir` 与实际文件名不符（用 `ref path` 比对）；(3) `years` 与文件年份不交叠；(4) `data_root` 指错根目录；(5) 站点 CSV 里的 `DIR` 是错误/相对路径。

22.7 其余 ref 管理命令

- `openbench ref register-profile NAME`: 注册”参考 profile” (变量名/单位映射模板), 供多个同类数据集复用。
- `openbench ref optimize NAME`: 把数据集转成 **Zarr** 以加速读取 (大数据反复评估时有用)。
- `openbench ref convert-old OLD.yaml OUT.yaml`: 把旧格式参考 YAML 转成 v3 registry 描述。
- `openbench ref delete NAME`: 删除用户 overlay 里的某条参考 (不影响包内置 catalog)。

22.8 注册与定制 model profile

模型 profile 描述某模式的变量名、单位与文件名约定。常用命令:

```
openbench model list                # 列出 22 个内置 + 用户 profile
openbench model show CoLM2024      # 查看变量映射
openbench model register MyModel ... # 注册/更新用户 profile
openbench model export CoLM2024 -o colm.yaml # 导出为独立 YAML (作模板)
openbench model import my_model.yaml # 导入用户 catalog
openbench model alias ...           # 建立别名
openbench model validate MyModel    # 检查完整性
openbench model status              # registry 状态 (内置/用户、完整度)
openbench model path MyModel        # 该 profile 是内置还是用户自定义
openbench model rename OLD NEW      # 重命名用户 profile
openbench model remove-var MyModel Latent_Heat # 从 profile 删一个变量
openbench model delete MyModel      # 删除用户 profile
```

配置里 `simulation.<label>.model` 引用 profile 名后, 变量文件名模式与单位换算自动套用; 条目内 `variables` 字段就地覆盖。对”按变量分文件”的模式, 用逐变量 prefix (见第三部分第 11 节与第四部分案例五)。

22.8.1 model profile 结构与示例

一个 profile 就是一份描述”该模式如何命名变量/文件/单位”的 YAML。顶层字段: `name`、`description`、`data_type`(grid/stn)、`tim_res`、`grid_res`、`data_groupby`、`time_offset` (可选), 以及 `variables` 映射 (每个标准变量 → `varname/varunit`):

```
MyModel:
  name: MyModel
  description: My land model historical run
  data_type: grid
  tim_res: Month
  grid_res: 0.5
  data_groupby: Month          # 文件按月切分
  variables:
    Latent_Heat:      {varname: LH,  varunit: W m-2}
    Sensible_Heat:    {varname: SH,  varunit: W m-2}
    Evapotranspiration: {varname: ET, varunit: mm s-1}
```

注册并验收:

```
openbench model import my_model.yaml      # 或 openbench model register
  MyModel ...
openbench model show MyModel              # 确认变量映射
openbench model validate MyModel          # 检查完整性
```

之后在配置里 `simulation.<label>.model: MyModel` 即可引用；文件名前后缀等可在 `profile` 里或 `simulation` 条目里给（条目覆盖 `profile`）。OpenBench 自动做单位换算（如 `mm s-1` → 评估单位）。

说明：快速起步：`openbench model export CoLM2024 -o my_model.yaml` 导出一个内置 `profile` 作模板，改名、改 `varname/varunit` 后再 `import`。不在内置 22 个里的模式都这样接入。

23 添加自定义指标、评分、统计与可视化

指标与评分分别定义在 `core/metrics.py` 与 `core/scores.py`。新增一个指标的约定：

- 在 `metrics` 类中新增方法，签名形如 `def my_metric(self, s, o)`（`s`= 模拟，`o`= 观测/参考，均为 `xarray.DataArray`）。
- 用 `xarray` 沿 `time` 维计算，并**显式处理边界**：除零、全 NaN、常数序列、样本过少等情形用 `xr.where(...)` 守卫，返回 NaN 而非崩溃。
- 成对剔除 NaN、保证 `s/o` 有效掩膜对齐；标准差/方差的 `ddof` 在相关指标间保持一致。

- 评分 (scores.py) 应归一化到 0–1 (越大越好), 便于进入 Overall_Score。

```
# core/metrics.py (示意)
def my_bias_ratio(self, s, o):
    num = s.mean(dim="time") - o.mean(dim="time")
    den = o.mean(dim="time")
    return xr.where(den != 0, num / den, np.nan)
```

新增后即可在配置的 metrics/scores 列表里按方法名引用。一致性约束 (如 ddof、掩膜对齐) 详见源码 core/metrics.py、core/scores.py。

23.1 新增后的测试与验证

新增指标/评分后, 至少做三件事再提交:

1) **边界值单测**——写一个真正可跑的 pytest, 覆盖正常/常数/全 NaN/样本不足, 断言要有意义 (不要写 `assert ... or True` 这种恒真断言)。下例用内置 correlation/bias 演示, 把它们换成你的新指标、并按其预期行为改断言即可:

```
# tests/test_my_metric.py
import numpy as np, xarray as xr
from openbench.core.metrics import metrics

def DA(values):
    return xr.DataArray(np.asarray(values, dtype=float), dims="time")

m = metrics()

def test_normal_inputs():
    s, o = DA([1, 2, 3, 4, 5]), DA([1.1, 1.9, 3.2, 3.8, 5.1])
    assert np.isfinite(m.correlation(s, o))      # 正常输入应得有限值
    assert m.correlation(s, o) > 0.9             # 高度相关

def test_constant_series_returns_nan():
    const, o = DA([2, 2, 2, 2, 2]), DA([1, 2, 3, 4, 5])
    assert np.isnan(m.correlation(const, o))      # std=0 -> 相关无定义 = NaN
    assert np.isfinite(m.bias(const, o))          # 但 bias 仍可算

def test_all_nan_returns_nan():
    nan, o = DA([np.nan] * 5), DA([1, 2, 3, 4, 5])
    assert np.isnan(m.bias(nan, o))              # 全 NaN -> NaN, 不抛异常

def test_insufficient_samples():
    s, o = DA([1, 2]), DA([1, 3])
```

```
np.asarray(m.correlation(s, o))          # n 很小也不应崩（返回值
/ NaN 均可）
```

跑: `python -m pytest tests/test_my_metric.py -q`。重点覆盖：除零（标准差=0/分母0）、全 NaN、常数序列、样本过少（ $n < 2$ ）、空数组——这些都应返回 NaN 而非抛异常。

2) 跑测试与 lint:

```
python -m pytest tests/test_core/ -q
ruff check src/ tests/
```

3) 确认缓存指纹变化：新增/改动 metrics/scores 源码后，其内容指纹应改变，从而旧缓存自动失效。验证方法：对同一配置先后 `openbench run`，改动源码后第二次应重算（而非命中旧缓存）；若没重算，检查改动是否在被指纹覆盖的模块内，必要时 `--force` 或删除 `.openbench_cache.json`。

23.2 添加自定义 statistics 与可视化

23.2.1 统计模块

统计方法位于 `core/statistics/`（每个 `stat_*.py` 一种方法，共 17 个）。新增时参照现有模块的输入/输出约定，随后在 `statistics.items` 中按名引用即可（注意 `statistics` 需 `enabled: true` 且显式列出 items）。

23.2.2 图件

图件位于 `visualization/Fig_*.py`。新增图种参照同目录现有实现，并在对比/报告流程接线。复用内置基础绘图（含 `cartopy` 底图封装）比手撸地图代码省很多时间；离线环境注意先备好 Natural Earth 底图（见第二部分第 5.2.2 节）。

23.3 自定义滤镜与数据处理

OpenBench 支持自定义数据滤镜（如对某参考做质量控制/筛选），放在 `data/custom/` 下，通过动态加载机制（`load_filter`）按名引用（内置示例如 `GEBA_filter.py`）。滤镜在预处理阶段对数据集生效，适合实现“只保留满足条件的样本/站点/时段”等逻辑，而无需改动核心代码。

24 远程执行、缓存与可复现

注意：远程执行目前仅通过 GUI 驱动，命令行尚未实现：`openbench run --remote` ... 会直接报错并提示改用 `openbench gui`。本节描述的是 GUI 远程流程与底层机制。

24.1 两种 HPC 用法

1. 直接在集群上跑（推荐、最简单）：把 OpenBench 装到登录/计算节点，像本地一样 `openbench run openbench.yaml`（或写进作业脚本由 Slurm/PBS 提交）。这条路不需要 `[remote]`。
2. 从本地经 SSH 提交到远端（GUI）：装 `[remote]` 后，用 `openbench gui` 配置远程主机、上传配置/数据、远端运行、回传结果。

24.2 在集群作业脚本里运行（Slurm 最小流程）

第 1 步：登录节点先自检（确认环境就绪，再提交作业，省得排队后才发现装错）：

```
source ~/openbench-env/bin/activate
openbench smoke-test --run          # 内置样例跑通 = 环境 OK
```

第 2 步：离线机器先备好 Cartopy 底图。计算节点通常无外网，绘图会因取 Natural Earth 失败而中断。在能联网的登录节点把底图放到共享目录，让计算节点复用（见第二部分第 5.2.2 节）：

```
export CARTOPY_DATA_DIR=/shared/cartopy
python -m cartopy.feature.download physical --output "$CARTOPY_DATA_DIR"
```

第 3 步：写作业脚本 `run_openbench.slurm`：

```
#!/bin/bash
#SBATCH -J openbench
#SBATCH -N 1
#SBATCH -n 16
#SBATCH -t 04:00:00
#SBATCH -o openbench_%j.log

source ~/openbench-env/bin/activate
export HDF5_USE_FILE_LOCKING=FALSE          # NFS/共享盘必备，避免文件锁
报错
```

```
export CARTOPY_DATA_DIR=/shared/cartopy      # 复用共享底图，离线也能成图
openbench run openbench.yaml --cores 16
```

第 4 步：提交：

```
sbatch run_openbench.slurm
```

说明：PBS/LSF 同理：把 #SBATCH 换成对应指令，正文三行（激活环境 + 两个 export + openbench run）不变。节点内存吃紧时减小 --cores 或在配置开 dask 并设 dask.memory_limit。

24.3 GUI 远程 ([remote])

remote 子包提供 SSH 连接 (connections/ssh)、凭据 (credentials)、远端存储与同步 (storage/sync)。在 GUI 的远程页中：配置主机名/端口/用户、选择密钥或密码、设置远端工作目录 → 上传配置与（可选）数据 → 远端运行 → 回传产物。

说明：排错要点：(1) 先用系统 `ssh user@host` 确认能免密登录（密钥已加入 `authorized_keys`）；(2) 远端需已装好同版本 OpenBench 与依赖；(3) 共享盘上设 `HDF5_USE_FILE_LOCKING=FALSE`；(4) 计算节点无外网时，预放 `CARTOPY_DATA_DIR` 底图（见第二部分第 5.2.2 节）。

大数据量评估可结合第三部分第 13 节的 dask 配置开启分布式调度，并按节点内存设 `num_cores` 与 `dask.memory_limit`。

24.4 缓存与可复现机制

24.4.1 两层缓存

OpenBench 有两层缓存：

- **增量评估缓存：**每个评估任务以“配置 + 算法版本 + 输入文件签名”为键缓存 (runner/hashing.py)。命中即跳过；源码 (metrics/scores 内容指纹)、配置或输入文件变化即自动失效重算。索引存于各输出目录的 `.openbench_cache.json`。
- **重网格权重缓存：**守恒重映射的权重跨 run 复用，存于全局缓存目录（可由 `OPENBENCH_REGRID_WEIGHT_CACHE_DIR` 指定）。

此外，输出原子写 + fsync，崩溃不留半截有效文件；后处理设闸门，任一任务失败则不产部分汇总；中断后重跑已完成安全跳过、未完成重做。

24.4.2 用 CLI 管理缓存

```
openbench cache status --regrid      # 查看重网格权重缓存状态
openbench cache status --regrid --json # 机器可读输出
openbench cache clear --regrid       # 清空重网格权重缓存
```

增量评估缓存按 run 存放，要清某次 run 的缓存，删除该输出目录下的 `.openbench_cache.json` 即可；或运行时加 `--force` 绕过缓存全量重算。

说明：怀疑“改了却没重算”时：(1) 确认改的是被纳入指纹的输入（配置/输入文件/算法源码）；(2) 开发态下若只改了源码而包版本号没变，靠 `metrics/scores` 内容指纹失效——若仍异常，直接 `--force` 或删除 `.openbench_cache.json`。

25 二次分析：用 Python 直接读结果

评估产物是标准 NetCDF / CSV，可直接用 `xarray` / `pandas` 再加工：

```
python - <<'PY'
import xarray as xr, pandas as pd

# 格点：逐格 metric NetCDF（维度 lat/lon 或 time/lat/lon）
ds = xr.open_dataset("output/<run>/metrics/<var>_ref_<R>_sim_<S>_KGE.nc")

# 站点：一行一站的评估 CSV
df = pd.read_csv("output/<run>/metrics/<var>_stn_<R>_<S>_evaluations.csv")

# 综合分数汇总（对比阶段，制表符分隔）
heat = pd.read_csv(
    "output/<run>/comparisons/HeatMap/scenarios_Overall_Score_comparison.csv",
    sep="\t")
print(ds, df.head(), heat)
PY
```

这样可把 OpenBench 的标准化产物接入自己的后续分析、绘图或论文流程。

26 参与开发

从源码安装开发环境（见第二部分）后：

```
python -m pytest -q          # 运行测试
ruff check src/ tests/       # 代码检查
python -m build               # 构建 wheel + sdist
```

贡献流程：在功能分支开发 → 跑通测试与 lint → 在 GitHub 提 Pull Request。新增功能应配套测试与文档；详见仓库 CONTRIBUTING。